Vivado 仿真与实现验证报告

★（2026-10-12 v12）前置说明：下文实现/时序结论已被 v12 重建取代

本报告正文的实现与时序数字均为 2026-10-10 的 v9 历史口径，不得作为"当前状态"引用。 2026-10-12 采纳 **v11** 后重建实现，**时序首次收敛**；当日晚间 RTL 升到 **v12**（`RTL_BUILD_ID = 16'h000C`，遥测帧布局 v2）并**已重新综合出生产版与 ILA 版位流**：

- XDC 修正：`constr/enc_top.xdc` 两处 `create_generated_clock`（`adc_clk` / `adc_clk2`）**去掉 `-invert`**（采样时钟与 `sys_clk` 同相），ADC 输入延迟由 `-max 24.000 / -min 19.500` 改为 **`-max 14.000 / -min 9.500`**（`constr/enc_top.xdc:219-222`，推导见 `:215-218`）；`rtl/enc_top.v:139` `RTL_BUILD_ID = 16'h000C`。
- **生产版（v12 现行）** `_synth_work/impl_timing.rpt` @2026-10-12 23:59:16：`sys_clk` **WNS +0.959 ns / TNS 0.000 / 0 个失败端点（11,275 端点）**，WHS +0.075、WPWS 9.500，结论行 **`All user specified timing constraints are met.`** ⇒ 产出 `_synth_work/enc_top.bit` **1,042,208 B @2026-10-12 23:59:27**，SHA256 `D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725`。
- **ILA 版（v12 现行）** `_synth_work/impl_ila_timing.rpt` @2026-10-13 07:11:57 结论行**同判据全绿**（WNS **+0.332** / 0 失败 / **20,165** 端点、WHS +0.050、WPWS 8.870）⇒ 产出 `_synth_work/enc_top_ila.bit` **1,428,064 B @2026-10-13 07:12:11**，SHA256 `5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`；`_synth_work/enc_top_ila.ltx` **71,337 B @07:12:00**（36 条命名探针 / 269 bit）。
- **v12 状态：已重综合出位流，待上板验证。**板上未换版时 `0x05F2` 仍可能读回 **`0x000B`**（v11）。
- **v11 = 历史（已归档）**：生产版 `_synth_work/enc_top_v11prod.bit` **1,086,600 B @2026-10-12 21:15:37**、SHA256 `FA1308647D776F78E2497DF44E92F06942749567800338 0A8D78D40F5B7C2594`（WNS +0.821 / 0 失败 / 11,310 端点、LUT 6369 / FF 4088）；ILA 版 `_synth_work/enc_top_ila_v11.bit` **1,458,744 B @21:29:54**、SHA256 `5E049FD3B3EAB51C79D62BEA35709D6CB922820064B316FE1C644DCB97B3210B`。
- **下文每一处 `Timing constraints are not met.` / `WNS -3.3xx`（以及 −3.352 / −3.329 / −3.274）都是 2026-10-10 v9 轮次的历史记录**，其根因 = ODDR / 生成时钟的 **`-invert` 相位不匹配**（`adc_clk` 上升沿 比 `sys_clk` 晚 10 ns ⇒ 捕获窗仅约 4 ns），已在 v11 去掉 `-invert` 并配套改输入延迟后消失。
- 位流大小历史（804,780 B @2026-10-10 16:05:29）与当前（1,042,208 B）不同**不是回归**：位流大小受压缩率 影响极大，判据看资源、时序与 DRC。

本文件记录**在本机用真实 Vivado 2026.1 工具链**对 `rtl/` 全部 RTL 做的仿真、 综合与实现验证。所有数字都来自实际运行的日志，不是期望值。

- 仿真器：Vivado Simulator (xsim) v2026.1
- 综合/实现器件：`xc7a35tfgg484-2`（正点原子 达芬奇-lite）
- 报告生成时间：2026-10-08

0. 一句话结论

19 个测试台全部 `TEST PASSED`（编译零 ERROR，`sim/logs/SUMMARY.md` 2026-10-10 16:21:05，19/19；16:03:11 那次同为 19/19）；RTL 综合 0 errors、0 critical warnings； ★（2026-10-12 v12）当前实现（place & route）时序已收敛：`All user specified timing constraints are met.` **（生产版 `_synth_work/impl_timing.rpt` @2026-10-12 23:59:16：`sys_clk` WNS = +0.959 ns / TNS = 0.000 / 0 个失败端点 / 11,275 端点，WHS = +0.075、WPWS = 9.500；ILA 版 `impl_ila_timing.rpt` @2026-10-13 07:11:57 同判据行，WNS +0.332 / 0 失败 / 20,165 端点）；** 以下是 2026-10-10 v9 历史口径：**实现（place & route）"Timing constraints are not met"（WNS = −3.352 ns、TNS = −85.116 ns、WHS = +0.033 ns、THS = 0、**26 failing endpoints 全部是 ADC 输入端口 **/ 10582 端点）；50/50 端口已绑定真实 `PACKAGE_PIN`，布线后 DRC 0 errors、0 critical warnings（43 条检查全是 `Related violations: <none>`），`write_bitstream` 成功产出 `_synth_work/enc_top.bit`（**（2026-10-12 v12）1,042,208 B @ 2026-10-12 23:59:27；2026-10-10 v9 历史为 804,780 B @ 16:05:29；v11 历史为 1,086,600 B @21:15:37，已归档 `_synth_work/enc_top_v11prod.bit`），头部含 `COMPRESS=TRUE`；生产版签核数字见** §12.4（2026-10-10 历史）与前文 v12 前置说明（现行），**含 14 个输出侧约束、§1.18 幂等标定修复、§1.20 探针位序修复、§1.21 OTR 软清位与 §1.22 UART→SPI 桥）。★ v9 的负余量**不是回归：**08:40 那版 `+0.923 ns / 0` 失败 是在 ADC 输入侧**完全没有约束**时得到的；12:08 起按** 总输入延迟预算 19.5~24 ns**（3.762 源时钟 + 10.000 半周期 + 3.016 ODDR/OBUF +** 手册 `tPD` 2~6 ns **+ 1 线缆；注意 19.5~24 ns** 不是**手册 tPD）写了 `set_input_delay -max 24.000`（§1.19），那是** v9 的反相相位口径，**此后每一版都必然为负，**（2026-10-12 v11 起）去掉 `-invert` 并改用 `14.000/9.500` 后已全部收敛（v12 沿用），**判据与回退方案见 `docs/07` §1.8。同一份 RTL 的** ILA 调试版已在当前源码上重建**（GOLD 许可证；**2026-10-13 07:12:11 **为 v12 版，`_synth_work/enc_top_ila.bit`** 1,428,064 B，**SHA256 `5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`，`enc_top_ila.ltx`** 71,337 B @07:12:00，**时序同判据全绿；**v11 版 1,458,744 B @2026-10-12 21:29:54 已归档为 `_synth_work/enc_top_ila_v11.bit`；2026-10-10 16:12 版为历史，**产出 `_synth_work/enc_top_ila.bit`** 1,445,812 B @ 2026-10-10 16:12:01）+ `enc_top_ila.ltx`（67,006 B @16:11:50），那时 `sys_clk` 域** WNS −3.329 ns / 26 个失败端点（全部是 ADC 输入端口）/ 18010 端点**（当时 ADC 输入约束的后果，与生产版同源，见 `docs/DEV_STATUS.md` §1.19）、DRC 62 checks 全 `Related violations: <none>`（0 error / 0 critical）—— 见 §12.1。验证过程共发现并修复** 15 个真实 RTL 缺陷 **和** 9 个测试台缺陷** （第 14 个即 §11 第 16 行"一键标定不可重复"，2026-10-10 修复并加了逐位一致的回归判据 F11h；第 15 个即 §1.22 的 UART→SPI 桥 TX 仲裁 `tx_hold`/`tx_drive` 混用导致遥测帧被截断）；详见第 4、5 节；未解决的已知缺口见第 11 节。

> **`Bitstream compression saved N bits` 的正确读法（曾写反过，此处为准）**：这一行里的 `N` 是**被省掉的位数**，不是压缩后的位数。`write_bitstream` 的日志原文是 `Bitstream compression saved 9836960 bits`，配合该器件的**未压缩长度 `17,537,696 bit`**（= 2,192,212 B，由 XC7A35T 的帧结构固定），得压缩后 = `17,537,696 − 9,836,960 = 7,700,736 bit` = **962,592 B**，与实测文件长度 962,600 B 正好差 **8 B**（同步字 + 头部，不参与压缩统计）⇒ 逐字节自洽。**2026-10-10 v9 历史生产版（2026-10-10 16:05:29，含 UART→SPI 桥）**：`Bitstream compression saved 11099520 bits` ⇒ `17,537,696 − 11,099,520 = 6,438,176 bit` = **804,772 B**，实测 **804,780 B**（同样差 8 B）；**（2026-10-12 v12）现行生产版为 `_synth_work/enc_top.bit` 1,042,208 B @2026-10-12 23:59:27**（`Bitstream compression saved 9200096 bits` ⇒ `17,537,696 − 9,200,096 = 8,337,600 bit` = 1,042,200 B，实测 1,042,208 B，同样 +8 B 头部差）；**v11 历史为 1,086,600 B @2026-10-12 21:15:37（已归档 `_synth_work/enc_top_v11prod.bit`）**；ILA 版 v12 为 `_synth_work/enc_top_ila.bit` 1,428,064 B @2026-10-13 07:12:11（`saved 6113248 bits` ⇒ `17,537,696 − 6,113,248 = 11,424,448 bit` =

> 1,428,056 B，实测 1,428,064 B，同样 +8 B 头部差）。 ★ **位流大小不能当回归判据**：同一份 RTL 历史实测 726,924 / 962,600 / 804,780 B， 差异全部来自压缩率与实现随机性（`impl_check.tcl` 自 9e9aebc 起未改），**只看资源、时序与 DRC。** ILA 版同理（2026-10-10 15:14 版）： `17,537,696 − 6,055,840 = 11,481,856 bit` = 1,435,232 B，实测 1,435,240 B （同样差 8 B）； 早先 08:56 版的算式是 `17,537,696 − 6,153,312 = 11,384,384 bit = 1,423,048 B`、实测 1,423,056 B， 各版算式各自自洽（大小不同是因为各版 RTL/布局不同）。 ★ 早期版本把 5,873,568 当成"压缩后长度"写进了报告， 那个数字其实来自另一次运行、且位置放错，现已按上式更正。
>
> ✅ **引脚分配与 bitstream 已闭环（本报告最重要的一条结论）**： 厂商 `AD9238_carrier_pin_map.xls` 工作表 7 的三个列名原文是 `双AD9226/AD9238对应管脚 **XC7A35V2.0T**扩展口 J6 / J7 / J11对应管脚` —— **这三个口是另一块 "XC7A35V2.0T" 板的扩展口，不是达芬奇-lite 的排针。** 本板的 `J7` 是 `RGB_TFTLCD`（RGB-LCD + 触摸排座），而且本板**根本没有 `J11` 连接器** （FGG484 上的 `J11` 是 `VCCBRAM` 球）。 三列中只有厂商 `J7` 列落在真实排针上（39/39 全 bank 16），它对应本板的 `J6`； 但**厂商列的针号与本板 `J6` 排针一个都对不上（0/28）**， 所以引脚是**逐针量取** `_refs/vendor/Davinci_Lite_1V0.pdf` sheet 13 的矢量几何得到的， 再用模块说明书 p9 §6 的针表与载板 netlist 互证。 最终由 `python tools/pinout_verify.py`（7 项 A–G 全 PASS）与 `python tools/pinplan_check.py`（`RESULT: PASS`）自动校核。 详见第 8 节与 `docs/04_wiring_checklist.md`。

验证层次	工具	结果（仿真：2026-10-12 23:52 最新一次干净复跑（v12 RTL）；综合/实现：2026-10-12 v12 生产版）
逐模块仿真	`xvlog` + `xelab` + `xsim -runall`	**21/21 TEST PASSED**，0 ERROR（`sim/logs/SUMMARY.md` **2026-10-12 23:52:26**；`tools/lint_all.ps1` 逐文件 `xvlog` = **ERROR 0 / WARNING 0**；早前 19 台那轮为 2026-10-10 16:21:05）
顶层集成仿真	`tb_enc_top`	**TEST PASSED (81 checks)**，4194240 点角度比对零超差（含 TEST F 顶层标定集成路径、幂等判据 F11a–h 与 TEST D2 的 OTR 软清位端到端 9 条）
桥/遥测集成仿真	`tb_uart_spi_bridge`	**TEST PASSED (44 checks)**：遥测 80 帧 / 1440 字节 / 0 帧错，应答 10 条（UART→SPI 桥的 TX 仲裁，§1.22）
遥测帧布局 v2（v12 新增）	`tb_uart_scope` + `tools/check_uart_frame_fixture.py`	**TEST PASSED (96 checks)**：帧偏移 8/9/10/11 = `io_state` / `0xA2` / `io_flags` / `0x00`、位序、以及"旧布局 off9 = `amp_sq[23:16]` 恒 ≤ 0x80 ⇒ 永不与 `0xA2` 撞车"的定向检查；夹具守卫 **30 checks PASS**（实拍帧 `aa 55 12 34 0f 0e f1 f2 6d a2 a6 00 01 02 03 04 5a ec` 逐字节比对 + "校验必须含帧头常量、不是纯载荷 XOR"的反事实检查）；`tools/uart_scope.py --selftest` PASS
模型对账	`model/golden_model.py` vs RTL	**RECONCILE PASSED**（0 / 1 LSB，门限 2 LSB）
综合	`synth_design`	**0 errors**，0 critical warnings

验证层次	工具	结果（仿真：2026-10-12 23:52 最新一次干净复跑（v12 RTL）；综合/实现：2026-10-12 v12 生产版）
实现	`opt_design` / `place_design` / `route_design`	**（2026-10-12 v12：已收敛 —— `All user specified timing constraints are met.`，WNS** +0.959 **/ TNS 0.000 /** 0 个失败端点 **/ 11,275 端点；ILA 版 WNS +0.332 / 0 失败 / 20,165 端点）**。下表为 **2026-10-10 v9 历史**：`Timing constraints are not met.`（WNS **−3.352** / TNS −85.116 / WHS +0.033 / **26 failed endpoints 全是 ADC 输入端口** / 10582 端点），含 14 个输出端口输出侧约束（§12.2）与 §1.19 的**总输入延迟预算口径**（19.5~24 ns = 源时钟 3.762 + 半周期 10.000 + ODDR/OBUF 3.016 + **手册 `tPD` 2~6** + 线缆 1；**不是手册 tPD**，且是 v9 反相相位值，v11 改 `14.000/9.500`）
引脚约束	`tools/pinout_verify.py` + `tools/pinplan_check.py`	**两个都 `RESULT: PASS`**（50/50，无重复、无冲突）
bitstream	`write_bitstream`	✅ **成功。（2026-10-12 v12）现行 `_synth_work/enc_top.bit` = 1,042,208 B @2026-10-12 23:59:27**（SHA256 `D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725`）；**v11 历史**：`_synth_work/enc_top_v11prod.bit` **1,086,600 B @2026-10-12 21:15:37**（SHA256 `FA1308647D776F78E2497DF44E92F069427495678003380A8D78D40F5B7C2594`）；**2026-10-10 v9 历史**：`enc_top.bit` **804,780 B**（2026-10-10 16:05:29，`COMPRESS=TRUE`；省掉 11,099,520 bit）

> ★ 本表**原为 2026-10-10 16:05 生产版状态**；**（2026-10-12 v12）现行状态为：时序收敛（`All user specified timing constraints are met.`，WNS +0.959 / TNS 0.000 / 0 个失败端点 / 11,275 端点），生产位流 1,042,208 B @2026-10-12 23:59:27，ILA 位流 1,428,064 B @2026-10-13 07:12:11。v11（历史，已归档）为 WNS +0.821 / TNS 0.000 / 0 个失败端点 / 11,310 端点、生产位流 1,086,600 B @21:15:37、ILA 位流 1,458,744 B @21:29:54。**历史轮次的数字（13/13 测试台、WNS +0.544、 741,836 B、9474 端点；P0-2 那一轮的 +0.512 / 731,612 B / 9487 端点；08:40 那一轮的 +0.923 / 962,600 B / 9631 端点）保留在 §9.1/§9.2/§12.2/§12.3 的"历史基线"小节里， 引用时务必标明是历史值 —— **尤其不要把 08:40 的 `+0.923 ns / 0 失败` 当成当前口径。★ （2026-10-12，RTL v12）源码又前进了一步**：遥测帧布局 v2（帧内带 ABZ/UVW 六路电平， 见 `docs/DEV_STATUS.md` §1.58 与 `rtl/SPEC_interface_contract.md` §1.13），已过 lint + 全量仿真回归； **并且已重综合出 v12 位流（生产版 2026-10-12 23:59、ILA 版 2026-10-13 07:12），待上板验证** —— 本文所有"现行"位流体积 / 时序 / 资源数字已随之更新为 **v12**（见前文 v12 前置说明）， v11 / v9 的数字一律标为**历史**；**板上未换版时 `0x05F2` 仍可能读回 `0x000B`（v11）。**

---

1. 工具环境

项目	值
Vivado	`D:\Vivado\2026.1\Vivado`，仿真器 `xsim`

项目	值
工具目录	`D:\Vivado\2026.1\Vivado\bin\{xvlog.bat, xelab.bat, xsim.bat, vivado.bat}`
PATH	环境变量里**没有** Vivado，每条命令前需 `$env:Path = 'D:\Vivado\2026.1\Vivado\bin;' + $env:Path`
`setupEnv.bat`	只设 `RDI_*` 变量，**不设 PATH**
许可	**GOLD**（`D:\Vivado\Xlnx_2026_1.lic`，含 `INCREMENT ChipscopePro ... License_Tier:GOLD`）。★ `D:\Vivado` 下**同时存在** `Xilinx.lic`（BASIC）与 `Xlnx_2026_1.lic`（GOLD），而 `XILINXD_LICENSE_FILE` 默认为空 ⇒ Vivado 命中 BASIC 并拒绝 `create_debug_core`（`ERROR: [Vivado 12-29205] ... Your current selected license is BASIC.`）。**必须显式** `$env:XILINXD_LICENSE_FILE = 'D:\Vivado\Xlnx_2026_1.lic'`，之后启动日志出现 `INFO: [Common 17-3922] A valid Vivado Design Suite GOLD license has been detected.`。`tools/run_vivado.ps1` 的 `-LicFile` 已默认指向它
沙箱注意	运行 `vivado` 前必须把 `HOME` / `USERPROFILE` / `APPDATA` 指向工作区目录，否则报 `ERROR: [Common 17-1219] Failed to create directory to save app.xml at 'C'`

关于 Xilinx 仿真库：本机没有预编译的 unisims 库（Vivado 2026.1 未随装 `compile_simlib`），但 `sim/xilinx_prims_stub.v` 为 `BUFG` / `ODDR` 提供了行为等价桩， 所以 `tb_enc_top` 可以直接仿真。综合时 Vivado 解析真实的 unisims 原语，与该桩无关。

---

2. 复原方式（别人照着可以复现）

```
# 0) 准备环境
$env:Path = 'D:\Vivado\2026.1\Vivado\bin;' + $env:Path
Set-Location 'C:\Users\zhaoy\Desktop\FPGA'

# 1) 跑全部 18 个测试台（单模块 TB 只编译被测模块；tb_enc_top 必须编译全部 rtl/*.v）
foreach ($tb in (Get-ChildItem sim\tb_*.v | ForEach-Object BaseName)) {
  $w = "_sim_work\reg_$tb"; New-Item -ItemType Directory -Force $w | Out-Null
  Push-Location $w
  xvlog --sv (Get-ChildItem ..\..\rtl\*.v | ForEach-Object FullName) `
         ..\..\sim\xilinx_prims_stub.v ..\..\sim\$tb.v
  xelab -debug typical $tb -s "${tb}_sim"
  xsim "${tb}_sim" -runall
  Pop-Location
}

# 2) 综合
vivado -mode batch -nolog -nojournal -source tools/synth_check.tcl

# 3) 实现（含真实时序）
vivado -mode batch -nolog -nojournal -source tools/impl_check.tcl

# 4) 与黄金模型对账
python tools/reconcile_cordic.py
```

> `tb_enc_top` 单个约 **4 分钟 CPU**（约 2.8 亿拍），13 个顺序跑完约 20 分钟。

---

3. 功能仿真结果

> **交付基线（历史）**：以下结果全部来自一次**全新目录下的干净回归**（`_sim_work/FINAL/`，13 个测试台各自独立重编译，不复用任何历史 snapshot）。 汇总：**13/13 `TEST PASSED`；`xvlog` ERROR = 0、WARNING = 0；`xelab` ERROR = 0。 ★ 该轮状态已扩到 19 台（在 18 台基础上新增 `tb_uart_spi_bridge`：UART→SPI 桥 + 遥测 TX 仲裁），最近一次干净串行复跑 2026-10-10 16:21:05 = 19/19 PASS（16:03:11 那次同为 19/19），见 `sim/logs/SUMMARY.md` 与 `docs/DEV_STATUS.md` §1.22。** ⚠️ 本文数字是历史快照：当前（2026-10-11）的权威回归状态是 21 台，见 `docs/DEV_STATUS.md` §1 顶部。** 逐台原始日志已归档在 `sim/logs/tb_*.log`（UTF-8 带 BOM）。

3.1 逐模块测试台（13 个，全部通过）

测试台	覆盖内容	结果
`tb_ad9238_capture`	偏移二进制→补码、边界码 0x000/0x800/0xFFF、OTR 锁存与清除、复位	TEST PASSED (0 errors)
`tb_decim_avg`	抽取正确性、算术右移符号、饱和、valid 时序	TEST PASSED (0 errors)
`tb_sc_correct`	四类畸变校正效果、粗调移位、正负饱和、floor 语义	TEST PASSED (0 errors)
`tb_amp_monitor`	平方和、上下限比较、门限倒置边界	TEST PASSED (0 errors)
`tb_cordic_atan2`	全圆 512 点扫描、四象限边界、退化输入（x=0 / y=0）	TEST PASSED
`tb_loop_filter`	匀速斜坡 21 组 kp×ki、阶跃 21 组、kp=ki=0 直通、lag_limit、freeze	TEST PASSED
`tb_pole_counter`	正/反向扫描、跨零辨向、Z 附近抖动不得误计数、稀疏 valid	TEST PASSED
`tb_abz_gen`	边沿数=4×inter、A/B 正交与占空、Z 宽度 vs `z_width`、后分频、限频、极性。★ 该台把 `.z_window(1'b1)` **常开**（`sim/tb_abz_gen.v:129/307`）⇒ "Z 宽度 vs z_width" 只在此条件下成立；集成路径的 Z 脉宽曾被门控窗截成 40 ns（v8 已修，见 DEV_STATUS §1.44）。**★ v9 起判据/循环改为 `z_width + 1` 个边沿（`zw = 0…8`，跨零拍不计入步数），见 DEV_STATUS §1.49**	TEST PASSED
`tb_uvw_gen`	6 状态序列、等宽驻留、方向反转、相位偏移、频率告警	TEST PASSED
`tb_biss_c_slave`	行为级 BiSS 主机模型：产生 MA、按位采样 SLO、独立算 CRC6，200+ 帧	TEST PASSED
`tb_spi_slave`	SPI 模式 0、地址+数据读写、TW28 风格 32bit 包	TEST PASSED
`tb_reg_file`	寄存器读写、`DEVICE_ID` =0x0C、配置位到扁平输出的映射	TEST PASSED

测试台	覆盖内容	结果
`tb_enc_top`	**顶层集成**（见 3.2）	TEST PASSED (39 checks)

3.2 顶层集成测试台 `tb_enc_top`

激励模型：65536 点 `sin/cos` 查找表（振幅 ±1800 LSB12），按偏移二进制 （`LUT ^ 0x800`，见 `rtl/ad9238_capture.v:53-54`）驱动两路 ADC； 相位每拍 +1 LSB，一个输入电周期 = 65536 拍，一个机械圈 = 64 个电周期 = 4194304 拍。

```
[TEST A] power-on reset timing ......... ok
[TEST B] forward rotation
  FWD: A_edges=4096 (1st rev 2048, expect 4096), Z_pulses=1 (1st rev 0), Z_width_max=2
       ang_cmp=4194240 pts, max_dev=5 LSB16, out_of_tol=0, limiter_hi=0 clk
       cyc_cnt: min=-1 max=62 wrap_pos=63 wrap_neg=0
[TEST C] reverse rotation
  REV: A_edges=4096 (expect 4096), Z_pulses=1, ang_cmp=4194240 pts, max_dev=5 LSB16, out_of_tol=0
[TEST D] status / alarms / UVW
  UVW: illegal_codes=0 cycles, sector_transitions=1536, min_dwell=29 clk, warn_hi=0 clk, pairs_cfg=4
[TEST E] BiSS: slo_changes=27 over 200 master bit periods, frame_total_bits=41, bit_pos=36
 TEST PASSED  (39 checks)
```

读数解释（写报告可直接引用）

指标	实测	说明
角度比较点	2 × 4194240	正、反两向各 419 万个采样点
瞬时角度最大偏差	**5 LSB16 = 0.0275°**	含 CORDIC 量化 + 环路恒速滞后
超差点数	**0**	容差 600 LSB16（3.3°）
A 边沿/机械圈	**4096**	理论 `4·inter/(2·div)·64 电周期 = 4096`，**精确一致**
Z 脉冲/机械圈	**1**	与 `rtl/enc_top.v:40`「64 极对磁环转一圈」一致
UVW 非法码	**0 cycle**	合法码集合
UVW 扇区切换	**1536**	理论 `64 电周期 × 4 pairs × 6 扇区 = 1536`，**精确一致**
UVW 最小驻留	29 clk	远大于告警门限 12 clk（`rtl/enc_top.v:426`），无频率告警
AB 限频活跃拍	**0**	该配置下 `gap=1`，不触发限频

覆盖的完整数据通路： `AD9238 采集 → 抽取平均 → 六参数校正 → CORDIC atan2 → PI 环路滤波 → 极对计数/Z → ABZ 输出 → UVW 输出 → BiSS-C 从站`

---

4. 修复的真实 RTL 缺陷（12 个）

> 判定原则：每个问题先判定是 **RTL 缺陷** 还是 **测试台期望缺陷**， 用 RTL 头注释规格、`rtl/SPEC_interface_contract.md` 、或 `model/` 里已验证的黄金模型做依据； **没有为了让测试通过而放宽任何断言或期望值。**

4.1 `rtl/amp_monitor.v` —— 门限倒置时未实现"low 优先"

- **现象**： `tb_amp_monitor` TEST 5 报 `ok=0 low=1 high=1` （期望 `ok=0 low=1 high=0` ）。
- **根因**： `rtl/amp_monitor.v:92-94` 原为 `verilog wire w_low = (amp32 < amp_low_sq); wire w_high = (amp32 > amp_high_sq); wire w_ok = (~w_low) & (~w_high);` 文件头注释第 32-33 行承诺"门限写反时 `amp_ok` 恒 0 且 **amp_low 优先**"， 但 `amp_low_sq > amp_high_sq` 时两者同时为 1，"low 优先"没实现。
- **修复**（ `rtl/amp_monitor.v:96` ）： `wire w_high = (amp32 > amp_high_sq) & (~w_low);` —— 强制三态互斥。

4.2 `rtl/abz_gen.v` —— 索引差与同拍锁存争用（相位累加器永不推进）

- **现象**： `tb_abz_gen` 波形全死， `edge_cnt` 不增。
- **根因**： `assign d_fwd_raw = idx_tgt - r_idx;` 与时钟块 `r_idx <= idx_tgt;` 同拍争用， 差值恒 0。
- **依据**：黄金模型 `model/abz_uvw_model.py:169-175` 用**推进前**的 `cur_idx` 作差， 到 `model/abz_uvw_model.py:274` 才 `cur_idx = tgt` 。
- **修复**：新增 `reg [9:0] r_idx_prev;` （ `rtl/abz_gen.v:149` ）， 时钟块 `r_idx_prev <= idx_tgt;` （ `rtl/abz_gen.v:359` ）， 两式改用 `r_idx_prev` （ `rtl/abz_gen.v:239-240` ）。

4.3 `rtl/abz_gen.v` —— 越界位选导致 `step_val` 恒 x （决定性）

- **根因**： `wire [10:0] d_sgn;` 只有 11 位，而 `step_val = trig_ok ? d_sgn[STEPW-1:0] : ...` 中 `STEPW=12` ，**越界选取** `d_sgn[11]` ； Verilog 对越界位选返回 `x` ， 于是 `step_val` 恒 x、两累加器比较恒假、输出全死。
- **修复**（ `rtl/abz_gen.v:265` ）：改为显式符号扩展 `verilog assign step_val = trig_ok ? {{(STEPW-11){d_sgn[10]}}, d_sgn} : STEP_ZERO;` （原先写的是零扩展 `{{(STEPW-11){1'b0}}, d_sgn}` ，那会把 −1 变成 +2047。）

4.4 `rtl/abz_gen.v` —— 索引回绕注入假步进

- **根因**： `d_fwd_raw` / `d_rev_raw` 为 **10bit 无符号**（mod 1024）， 而索引周期是 `4*inter` ，每次回绕注入 500~1000 的假步进。
- **修复**：改为 `wire signed [10:0]` ，并用 `{{1'b0, idx_tgt} - {1'b0, r_idx_prev}}` 按完整周期 `4*inter` 双向折叠取最短路径。

4.5 `rtl/abz_gen.v` —— 限频保持计数器 off-by-one

- **根因**： `r_hold <= (gap16 == 0) ? 0 : gap16[LIMW-1:0];` 装载 `gap16` 而下一拍才开始倒数 → 实际间隔 = `gap16+1` 。临界配置 inter=256/div=8 下 自然间隔正好等于 `gap16` ，于是每隔一次被挡掉，输出率腰斩为 64（应为 128）。
- **依据**： `rtl/abz_gen.v:45` `gap_clk = ceil((20·(ablimit+1)·div)/CLK_PERIOD_NS)` ； 用黄金模型独立算出该配置恰好 128 个 A/B 事件。
- **修复**（ `rtl/abz_gen.v:392` ）： `verilog r_hold <= (gap16 <= 16'd1) ? {LIMW{1'b0}} : (gap16[LIMW-1:0] - {{(LIMW-1){1'b0}}, 1'b1});`

- 修后 TEST D 三行 `min_gap` = 20 / 88 / 245，门限 `th_gap` = 16 / 84 / 244。

4.6 `rtl/abz_gen.v` —— 极性输出分支恒为正（零扩展丢符号）

- **现象**：TEST E 六种极性配置 `edge_cnt` 全为 +249（应正负交替）。
- **根因**：零扩展（见 4.3）使 `dir_rev` 分支永远发不出负步进。
- **修复**：同 4.3 的符号扩展。
- **修后**：正常 +249、`apol=1` −249、`bpol=1` −249、`both=1` +249、 `abzdir=1` −249、`apol+abzdir=1` +249，与 RTL 头注释引用的 **TW28 §6.1** 一致。

4.7 `rtl/pole_counter.v` —— `prev_ang` 与 `ang_d1` 同拍更新（跨零检测失效）

- **根因**：`ang_d1 <= angle;` 与 `if (vld_d1) prev_ang <= ang_d1;` 同拍推进， 使 `prev_ang` 实际比 `ang_d1` **早一整拍**（相差 2 个样点），判据永远看"过时"窗口； 实测反向扫描一个跨零都检测不到。且原判据本质上不对称， `prev=0/cur=315`（越过 315° 的大正跳变）被误判为负跨零。
- **修复**：重排为 `ang_s0`（第 0 级 = 本拍 angle）与 `ang_s1`（第 1 级 = 上一有效样点） 级联，判据改为"相邻两个有效样点是否跨越顶窗/底窗 + 步长门限"： `verilog wire [AW-1:0] step_fwd = ang_s1 - ang_s0; wire [AW-1:0] step_bwd = ang_s0 - ang_s1; localparam [31:0] STEP_LIM = (32'd1 << (AW - 2)); // 90° localparam [31:0] STEP_MIN = (32'd1 << AW) - (32'd1 << (AW - 2)); // 270° wire step_big_f = (step_fwd >= STEP_MIN); wire step_big_b = (step_bwd >= STEP_MIN); wire pair_valid = have_prev & vld_d1; wire in_top = (ang_s0 >= TOP_TH) | (ang_s1 >= TOP_TH); wire in_low = (ang_s0 <= LOW_TH) | (ang_s1 <= LOW_TH); wire spans_zero = in_top & in_low; wire wrap_p_raw = pair_valid & spans_zero & step_big_f; wire wrap_n_raw = pair_valid & spans_zero & step_big_b;` ★ 坑：`STEP_MIN` **必须**写成 32 位常量；写成 `wire [AW-1:0] STEP_MIN = (1<<AW)-(1<<(AW-2));` 会在 16 位上下文里把 `1<<16` 截成 0 而**静默失效**。

4.8 `rtl/pole_counter.v` —— 负跨零用了错误的方向极性

- **根因**：`wrap_n_raw = ... & step_small_b;`，而 `step_bwd = 2^AW - step_fwd` ⇒ `step_small_b ≡ step_big_f`，导致一次真实正跨零**同时**触发两个方向 （探针实测 `step_f=65024 bigf=1 smlb=1 | wp_ev=1 wn_ev=1`），而真实负跨零什么都不触发。
- **修复**（`rtl/pole_counter.v:128`）： `wire wrap_n_raw = pair_valid & spans_zero & step_big_b;` 依据是 `rtl/pole_counter.v:15-16` 的原文。

4.9 `rtl/pole_counter.v` —— `pos_full` 低位配错样点

- **根因**：`wire [CW+AW-1:0] pos_nxt = {cnt_nxt, ang_s1};` 跨零由 `ang_s0` 触发、本拍寄存，故该拍应与 `cnt_nxt` 配对的是 `ang_s0`； 用旧的 `ang_s1` 会把新计数配上**上一个**样点的角度。 修前探针跨零行 `out_valid=1 cnt=1 pos_full=130560` → 低半 65024（357.2°）而非 0°。
- **依据**：`model/pole_counter_model.py:73` 用最新样点角度，同一约定。
- **修复**（`rtl/pole_counter.v:154`）： `wire [CW+AW-1:0] pos_nxt = {cnt_nxt, ang_s0};`
- 修后 170 周期探针逐拍零失配，跨零行 `cnt=1 pos_full[15:0]=0 wp=1 wn=0`（平滑 360°）。

4.10 `rtl/enc_top.v` —— POR 计数器无初值，xsim 下全芯片恒 X （致命）

- **现象**：顶层集成仿真首个运行里 `enc_top` 内**每一个寄存器都是 X**： `status: otr=x amp_low=x ... led=0xxx irq_n=x`、 `BiSS: frame_total_bits=x`。

- **根因**（`rtl/enc_top.v:99-100` 原为）： `verilog reg [19:0] por_cnt; // 无初值、无复位 wire rst_n = por_cnt[19]; // X → 全芯片永远不复位` 注释只声称"Xilinx FDRE 上电时 INIT=0，因此 `por_cnt` 从上电起自由计数" —— 这是**器件特性**，xsim **默认不建模**它，于是 RTL 文档所述行为与仿真行为不一致， RTL 事实上**无法被验证。**
- **修复**（`rtl/enc_top.v:112`）： `verilog reg [POR_BITS-1:0] por_cnt = {POR_BITS{1'b0}};` 保留自启动语义；综合等价于 FDRE INIT=0（真实器件行为不变）。
- **修后**： `status: otr=0 amp_low=0 amp_high=0 amp_ok=1 cor_sat=0 ... led=0100 irq_n=1`， `por_cnt[19]` 在 force 释放后 1 拍正确拉高，TEST A/TEST E 全过。

4.11 `rtl/ad9238_capture.v` —— OTR sticky 清除只赢一拍

- **根因**：清除请求只对被清的那一拍有效，2 级同步器里仍携带"清除前"电平， 下一拍 `else if (otr_now)` 又把它重新置位。
- **修复**：新增 `wire otr_flush = clr_otr | en_fall;`（`rtl/ad9238_capture.v:92`）， 清除时**冲刷整条 OTR 同步流水线**（`rtl/ad9238_capture.v:109`），数据同步保持无条件。

4.12 `rtl/spi_slave.v` —— `ss_fall`/`ss_rise` 极性写反

- **根因**： `ssn_s3` 是 `ssn_s2` 延迟 1 拍，原写法 `ss_fall = ssn_s2 & ~ssn_s3` 检出的其实是**上升沿。**于是包模式在帧开始、 `sr` 刚被清零时就去解析命令， `sr[31]` 恒 0（28 个错误全部由此而来）。
- **修复**（`rtl/spi_slave.v:105-106`）： `verilog wire ss_fall = ~ssn_s2 & ssn_s3; // 片选选中（ss_n 下降沿，帧开始） wire ss_rise = ssn_s2 & ~ssn_s3; // 片选释放 -> 命令生效（帧结束）`

> 另有两处 **测试台缺陷**（不是 RTL）： `sim/tb_spi_slave.v:326` 把 `16'h0083` 写成应为 `16'h00C3`（bit6=abzen，见 `rtl/reg_file.v:67` 地址表与 `rtl/reg_file.v:483`）； 以及漏写 `0x010B`（UVWPH）包。 `rtl/reg_file.v` **未改一行。**

4.13 `rtl/uvw_gen.v` —— `eang*6` 因位宽不足被截断成 `eang*5` （扇区几何，新发现）

- **根因**（`rtl/uvw_gen.v:103-107`）： `verilog wire [AW+2:0] eang_x6; assign eang_x6 = {eang, 2'b00} + {eang, 1'b0}; // 本意 eang*4 + eang*2 = eang*6 wire [2:0] sec_raw; assign sec_raw = eang_x6[AW+2:AW]; // 0..5` 两个拼接操作数都只有 `AW+2 = 18` 位， `eang_x6` 也只有 18 位 ⇒ 相加结果被截断成 `(eang*6) mod 2^18`，而 `[AW+2:AW]` 实际取到的是 **`eang*5`** 的高 3 位。
- **影响**：65536 个角度里 **32768 个（50.0%）扇区号偏小 1**；真实扇区边界从 `10923 / 21846 / 32768 / 43691 / 54614` 变成 `13108 / 26215 / 39322 / 52429`； `sec_raw` 只用到 0..4（永不出现 5），并且每个输入周期在 4→5→6→7→0 处因 `step_f > 1` 的"补一步"分支（`rtl/uvw_gen.v:151-155`、 `:157-162`）多出 **1 拍毛刺态。**
- **为什么此前的测试台结构上不可能发现它**：原 TEST 1/2 只统计"6 次状态切换"与 "6 状态驻留极差 ≤ 2"。截断后的**平均驻留仍是 ~10923 拍**（独立 Python 复刻 RTL 状态机 实测：截断版 `{0:[10923,10919], 1:[10923], 2:[10922], 3:[10923], 4:[1], 5:[1], 6:[1], 7:[1]}`， 修复后 `{0:[10923], 1:[10923], 2:[10922], 3:[10923], 4:[10923]}`），所以"极差 ≤2"照样通过。
- **修复**（位宽显式、语义自证， `rtl/uvw_gen.v:103-111`）： `verilog wire [AW+2:0] eang_x6; assign eang_x6 = ({3'b0, eang} << 2) + ({3'b0, eang} << 1); // eang*4 + eang*2 = eang*6` 宽度依据： `6*65535 = 393210 < 2^19 = 524288`， `AW+3 = 19` 位足够容纳。
- **验证**： `sim/tb_uvw_gen.v` 新增 TEST 9，逐角度扫描**全部 65536 个点**，记录 `sec_raw` 每次跳变的角度，实测 5 个边界**精确等于** `ceil(k*65536/6)` = `10923 / 21846 / 32768 / 43691 / 54614`， 且 `sec_raw > 5`

的拍数为 0、跳变次数恰为 5。判据里"理论边界"由 RTL 的定义**现算**（`(k*65536 + 5) / 6`）而不是硬编码，避免手算误差掩盖问题。

- **★ 两次失败的修法尝试（教训）**：① `{3'b0, eang} + {{(AW-14){1'b0}}, eang, 1'b0}` 实测得到 **`eang*3`**（`eang=8192 → x6=24576`）；② `{1'b0, eang, 2'b00} + {2'b00, eang, 1'b0}` 同样得到 `eang*3`。**结论**：这种"拼接 + 相加"表达式极易搞错位宽与权重，必须用 `({3'b0, eang} << 2) + ({3'b0, eang} << 1)` 这种显式左移形式，**并且一定要用实测数值验证，不能只看代码推演。**

4.14 `rtl/uvw_gen.v` + `rtl/enc_top.v` —— `cfg_hyst_uvw` 未接线（缺口闭合）

- **原状**：`rtl/reg_file.v:472` 已经导出 `assign cfg_hyst_uvw = r_uvw_hyst[4:0];`（寄存器 `0x0104`，复位值 `rtl/reg_file.v:230` `8'h00`），`rtl/enc_top.v:457` 也声明了 `wire [4:0] cfg_hyst_uvw`，但 **`uvw_gen` 实例上没有这个端口** —— 因为 `rtl/uvw_gen.v` 原先根本没有该输入，迟滞完全由 elaboration 参数 `UVW_HYST_SHIFT` 决定。`rtl/enc_top.v` 还留了一段注释明说这是"本版本的一个明确缺口"。
- **修复**：
- `rtl/uvw_gen.v` 新增 `input wire [4:0] uvw_hyst,`，并把迟滞赋值改为 `verilog wire [AW-1:0] hyst_lsb; assign hyst_lsb = (uvw_hyst != 5'd0) ? {{(AW-5){1'b0}}, uvw_hyst} : (1 << (AW - UVW_HYST_SHIFT));`
- `rtl/enc_top.v` 实例接线 `.uvw_hyst (cfg_hyst_uvw)`；原"已知未接线项"注释整段改写，并把 `cfg_hyst_uvw[0]` 从 `wire unused_ok = &{...}` 汇聚列表里**删掉**（否则与端口驱动冲突）。
- `sim/tb_uvw_gen.v` 新增 `reg [4:0] uvw_hyst_cfg;` 与 `.uvw_hyst (uvw_hyst_cfg)`，并在 `cfg` 任务里默认置 0，保证既有 TEST 1–8 行为不变。
- **语义取舍（为什么这样定）**：`0x0104 != 0` 时**该值直接作为角度 LSB 死区**（1 LSB = 360/65536 = 0.0054932° 电角度，5 位上限 31 LSB = 0.170°）；`0x0104 == 0x00` 时**回退到编译期默认 `UVW_HYST_SHIFT`**（默认 6 → 2^(16−6) = 1024 LSB = 5.625°）。这样既让寄存器真正可用，又让**未驱动该端口的旧实例与旧测试台行为完全不变**（Verilog 未连接输入默认 0 ⇒ 自动回退）。
- **验证**：`sim/tb_uvw_gen.v` TEST 9 实测 `hyst_lsb = 1024`（配 0）与 `= 31`（配 31）；边界穿越用例 `eang=5000 → r_sec=0`、`eang=32768 → r_sec=3`、`eang=32793`（扇区内 +25）`→ r_sec=3`。
- **~~如实写明的既有局限**：`rtl/uvw_gen.v:156` / `:163` 是 `nxt_bnd = nxt_sec * SECT_W;`，而 `SECT_W = 1 << (AW-3) = 8192`，所以 `r_bnd` 落在 **8192 的粗栅格**上，`d_fwd = eang - r_bnd` 一旦跨扇区就 `>= 8192`，**远大于 5 位死区上限 31 ⇒ 该死区只在"`eang` 刚好贴着 8192 栅格边界"的极窄角度窗内真正起作用**，对"在真实扇区边界 54614 附近的抖动"实际上不构成保护。~~ ⇒ ✅ **该局限已于 2026-10 修掉，见 §4.15。**
- **TW28 规格差异**：手册的 `UVW.hyst = ±hyst×360/2048` 与本实现的"角度 LSB"单位不同，已在 `rtl/uvw_gen.v` 头注释里写明。

4.15 `rtl/uvw_gen.v` —— 迟滞基准由 8192 粗栅格改为真实扇区边界（原局限已消除）

缺陷是什么

§4.14 修好 `uvw_hyst` 接线后，`0x0104` 写 1…31 已经能改变 `hyst_lsb`，**但死区实际不起作用**：

```
// 旧实现（rtl/uvw_gen.v 的扇区推进分支）
nxt_bnd = nxt_sec * SECT_W;          // SECT_W = 1 << (AW-3) = 8192
```

`r_bnd` 因此落在 **8192 的粗栅格**上，而不是真实扇区边界。 `AW=16` 时真实扇区宽是 `65536/6 = 10922.67`，与 8192 每扇区差 `2730.67 LSB`。 于是 `d_fwd = eang − r_bnd` 一旦跨扇区就 `>= 8192`，**远大于 5 位死区上限 31** ⇒ 死区只在"`eang` 刚好贴着 8192 栅格边界"的极窄角度窗内真正起作用， 对"在真实扇区边界 54614 附近的抖动"完全不构成保护。

修法：真实边界基准

```
// ★ 真实扇区边界 BND[k] = ceil(k * 2^AW / 6)（k = 0..6）
//   AW=16: 0 / 10923 / 21846 / 32768 / 43691 / 54614 / 65536
function [AW:0] bnd_of;
    input [2:0] kk;
    begin
        case (kk)
            3'd0:    bnd_of = 17'd0;
            3'd1:    bnd_of = 17'd10923;
            3'd2:    bnd_of = 17'd21846;
            3'd3:    bnd_of = 17'd32768;
            3'd4:    bnd_of = 17'd43691;
            3'd5:    bnd_of = 17'd54614;
            default: bnd_of = 17'd65536;   // 3'd6: 一个输入周期
        endcase
    end
endfunction
```

迟滞判据改为**两个方向各对各自主边界**，语义对称：

```
assign off_fwd = ... ;                                          // eang_x − BND[r_sec+1]
assign off_bwd = $signed(bnd_bwd_nxt) - $signed({1'b0, eang_x});    // BND[r_sec] − eang_x
// ≥ H ⇒ 越过该边界后再走出去 H
```

死区以**刚跨过的那条真实边界**为中心，前后各 `hyst_lsb` LSB 对称且方向可逆。

修此缺陷时定位到的另外 3 个真缺陷

#	缺陷	证据与修法
1	**冷启动基准取错。**旧冷启动取"角度所在段的两条边界"`r_fwd = seg_lo` / `r_bwd = seg_hi`。若开机角度落在某段**高端**，`seg_hi` 已是段上界 = 下一条边界 ⇒ 第一次前向提交要求 `eang >= seg_hi + H`，而 `seg_hi` 就是那条边界 ⇒ **本该在 `BND+H` 提交的点永不满足**	改为**稳态语义**（与"上一拍刚提交完"完全等价）：`r_fwd = BND[sec_raw+1]`、`r_bwd = BND[sec_raw]`，第一拍判据就与稳态同一门限
2	**判据引用的基准比 `r_sec` 晚一拍。** `r_fwd` 与 `r_sec` 虽在同一 `always` 块非阻塞赋值，但 `r_fwd` 又**组合参与** `off_fwd` 求值 ⇒ 提交那一拍仍用**旧基准**（正是刚被越过的那条）⇒ `off_fwd >= H` 保持 ⇒ 下一拍**再提交一次**，一个边界跳两个扇区	判据改用与 `r_sec` **同拍**的组合基线 `bnd_of(r_sec+1)` / `bnd_of(r_sec)`，使"比较用基准" == "提交后生效基准"。逐拍证据（隔离探针）：`cnt=736 sec=0→1 mf=1`、`cnt=737 sec=1→2 mf=1`（应为 0）、`cnt=738 sec=2` ⇒ 一跳两格

#	缺陷	证据与修法
3	**扇区 5→0 的环绕被吞掉。** `BND[6] = 65536` 等于一整圈，在 16 位角度里退化成 0，绕回低半圈后 `off_fwd = eang - 65536 ∈ [-65536,-1]` 恒为负 ⇒ 前向判据永不成立，扇区**卡在 5**	把这**唯一一条**环绕边界写成双分支：`r_sec == 5 && eang_x < BND[1]` 时 `off_fwd = eang_x`（"绕回后的推进量"），使 5→0 提交点与其它扇区的 `BND[k]+H` 同构

另有 1 处**纯观测型**修正：`r_bnd`（只作探针/文档观测量，**不参与任何逻辑**）在反向提交时 记为"新扇区上边界" `bnd_of(nxt_sec_b + 1)`，与前向 `bnd_of(nxt_sec_f + 1)` 几何对称 —— 两者都等于**刚跨过的那条真实边界。**

验证

`sim/tb_uvw_gen.v` TEST 9 全 65536 点扫描：

```
  扇区边界实测: 1->10923  2->21846  3->32768  4->43691  5->54614
  （与 ceil(k*65536/6) 逐项一致）
TEST PASSED
```

- 前向/反向切换点**对称**落在 `BND[k] ± H`（`H = 31`）：前向 `10954/21877/32799/43690/54645`、 反向 `10892/21815/32737/43660/54583`（`= BND[k] - 31`，逐项相等）。
- 死区内抖动**零切换。**
- 全量回归回到 **13/13**（`sim/logs/SUMMARY.md`）。

★ 教训（写给下一次改这个状态机的人）

- **迟滞判据的基线必须与提交寄存器同拍。**任何"寄存器存基线 + 组合比较"的写法都会引入 "提交后旧基线仍满足"的连跳，除非比较用的就是即将锁存的值。
- **冷启动基准必须等于稳态语义下的那个值**，不能取"角度所在段的两条边界"。 此前十一版反复失败的根源都在这两条上，**而不是在"判据公式"上。**
- `BND[6] = 65536` 是 17 位；进入 16 位表达式前必须显式处理环绕，否则它退化成 0。
- 唯一的有效调试手段是**隔离探针** `_sim_work/probe_iso.v`（一个 `uvw_gen` + 一个独立写出的 参考模型，逐拍比对 `r_sec` 并输出事件轨迹）：不依赖 `sim/tb_uvw_gen.v`、不依赖 `enc_top`， 单台迭代约 10 s；在它之上定位上述 3 个缺陷各只花了几轮。

---

5. 修复的测试台缺陷（7 个）

测试台	缺陷	修复
`tb_loop_filter`	稳态统计窗口开在循环**之前**，统计到的是上电暂态	移到 `t == n - TAILW`（`:203`）与 `t == n - 500`（`:244`）
`tb_loop_filter`	阶跃测试只跑 4000 拍，而 ki=3&kp=6 需 ~2356 拍才进带，尾窗仍有残差	`n = 4000` → `n = 6000`（**判据阈值 128/20972 一字未改**）
`tb_cordic_atan2`	监视线在 DUT 寄存 `angle` 的**同一 posedge** 用 `out_valid` 采样，非阻塞语义读到**前拍** `angle` → 511/512 点恒定 −128 LSB 偏置	新增 `reg vld_d; vld_d <= out_valid;` 作采集限定；`lat_meas = lat_cnt + 1` → `lat_cnt`

测试台	缺陷	修复
`tb_uvw_gen`	`integer dwell [0:5]` 却用**原始** 3bit 码索引（合法码 1..6）→ 状态 110 **写越界**、下标 0 永不写入 → `dwell_lo` 恒 0	改为 `dwell [0:7]` 按原始码索引
`tb_uvw_gen`	DUT 输出有 2 clk 延迟，窗口却开在输入周期边界 → 首样本是上一电周期的陈旧尾巴	新增 `SW_OFF = 1` 对齐窗口；`prev_u` 用真实 U 电平播种；驻留极差改为完整电周期内统计
`tb_uvw_gen`	TEST 8 的 `4096 × 32 mod 65536 = 0` → eang **恒 0**（伪装的直流），`sec_changed` 永不置位	改 `cfg(5'd0, …)`（`pairs_eff` 使 5'd0 ≡ 32 极对）+ `sweep_slope(2000, 341)` → 1997 个告警拍
`tb_pole_counter`	`pair` task 在脉冲**只被组合呈现**的那一拍就读，而 DUT Latency = 2 clk → 每次都读到 0	读取前多等一个 `@(posedge clk);`
`tb_abz_gen`	探针初值 `pa = 1'bx;` 使 `pa !== 1'bx` 为真，采样点 0 被当成幻影边沿	改为 `pa = a_out; pb = b_out;`
`tb_abz_gen`	把**有效分频值**写进 `div` 端口，而契约是 `div = 寄存器值 + 1`	改 `dv[k][2:0] - 3'd1`（断言与容差未动）
`tb_reg_file`	3 处期望值错（`cfg_div` 是原始字段、漏 0x010B 包、`8'hC3` 应为 `8'hC7`）	逐条修正，`rtl/reg_file.v` 未改

6. 与黄金模型对账（`sim/README.md:100-109` 的强制步骤）

`tools/reconcile_cordic.py` 用**三角验证**：

1. `sim/tb_cordic_atan2.v` 内含三张 256bit 打包表（各 32 块 × 16 = 512 点）： `xtab`/`ytab`（理想 sin/cos 输入）、`etab`（期望角度）。
2. 该测试台在真实 xsim 下已判"黄金表逐位比对: **不符 0 点**、最坏 |误差| **0 LSB**" ⇒ **RTL 输出 === etab 逐位。**
3. 本脚本把 `model/golden_model.py` 的定点 CORDIC 跑在**同一组 (x,y)** 上， 再与 `etab` 比较。

```
[1] tb 的 etab 是否等于输入向量的几何角度 atan2(y,x)
    max |etab - atan2(y,x)| = 0 LSB    （不符 0 点）
[2] 黄金模型 vs etab
    max |err| = 1 LSB = 0.005493 deg
    不符点数  = 86 / 512
    RMS |err| = 0.4098 LSB = 0.002251 deg
[3] 三角闭环
    RTL === etab  (xsim 实测, 0 点不符)
    GM  === etab  (本脚本, 86 点不符, 最坏 1 LSB)
    => RTL === 黄金模型，最大差 1 LSB（门限 2 LSB）：通过
 RECONCILE PASSED
   (a)=max 0 LSB, (b)=max 1 LSB
```

> ★ 注意：`sim/tb_cordic_atan2.v:326` 的注释写"理想角度 = k×128 LSB"，但实测基准表是**从 15°(1920 LSB) 起步、每步 +128 LSB**（输入向量反证：`x[0]=32213≈32767·cos15°`、`y[0]=5998≈32767·sin15°`）。按注释写解析式会 误报 480 点不符。**该注释应修正**（不改也不影响测试台判决，因为测试台是 拿 `etab` 直接比对，而 `etab` 与输入向量几何角度逐位一致）。

`model/golden_model.py` 本身实测数字（可直接进报告）：

指标	数值
复合畸变校正前 → 后（三参数）	20.9839° → **0.0165°**（改善 **1273 倍**）
复合畸变校正前 → 后（含二次谐波 + ADC 噪声）	21.0773° → 0.3571°（65.0 LSB），RMS 0.1855°
CORDIC 全圆扫描	max |err| = 1.00000 LSB = 0.005493°，RMS 0.44180 LSB
CORDIC 8 个正交边界点	误差 **0.00**（359.99° 处 +0.0045°）
抽取 4 次的等效 ENOB 提升	+1.00 bit（12bit ADC → 约 13bit）
P=64 时的机械角分辨率	0.309 arcsec

---

7. 综合与实现后的资源占用

7.1 综合阶段（`tools/synth_check.tcl`，Device State = Synthesized）

```
Synthesis finished with 0 errors, 0 critical warnings and 46 warnings
```

资源	综合后占用	占比
Slice LUTs	5737 / 20800	27.58%
Slice Registers	3475 / 41600	8.35%
DSPs（DSP48E1）	16 / 90	17.78%
Bonded IOB	48 / 250	19.20%
BUFGCTRL	1 / 32	3.13%

> 注：`Bonded IOB = 48` 与 `enc_top` 的端口数 48 完全相等 —— 每个端口都已绑定真实 `PACKAGE_PIN`（第 8.2 节）。

7.2 实现阶段（`tools/impl_check.tcl`，Fully Routed）

> **★ 本表是 2026-10-10 之前（输出侧约束补齐前）的一轮实测快照（LUT 5653 / FF 3475 / WNS +0.544）。 (2026-10-12 v12）当前签核见前文 v12 前置说明**：生产版 WNS **+0.959 ns** / TNS 0.000 / **0 个失败端点** / 11,275 端点， 位流 **1,042,208 B @2026-10-12 23:59:27**，资源 LUT **6298**（30.28%）/ FF **4074**（9.79%）/ DSP 16 / IOB 50 / BUFGCTRL 1（`impl_util.rpt` @23:59:14）；**v11（历史，已归档）**：WNS +0.821 / 0 失败 / 11,310 端点、LUT 6369 / FF 4088、位流 1,086,600 B @21:15:37； 结论行 `All user specified timing constraints are met.`； **2026-10-10 v9 历史状态见**

> **§12.4**：LUT **6010（28.89%）** / FF **3857（9.27%）** / WNS **−3.352 ns**（P0-2 那一轮是 LUT 5656 / FF 3476 / WNS +0.512 ns，见 §12.2）。 下表保留以说明"两个 LUT 数字都正确、不要混用"这条经验。

资源	实现后占用	占比
Slice LUTs	5653 / 20800	27.18%
Slice Registers	3475 / 41600	8.35%
Slice	1742 / 8150	21.37%
LUT as Logic	5554 / 20800	26.70%
DSPs	16 / 90	17.78%
Bonded IOB	49 / 250	19.60%
BUFGCTRL	1 / 32	3.13%
CARRY4	984	—
Block RAM Tile	0 / 50	0.00%

> **两个 LUT 数字都正确、不要混用**：§7.1 的综合结果是 5737（`tools/synth_check.tcl` → `_synth_work/util.rpt`），§7.2 的实现结果是 **5653**（`tools/impl_check.tcl` → `_synth_work/impl_util.rpt`）。 差值是 opt/place 阶段的逻辑优化与 `IOB` /LUT 重新映射造成的正常现象。 与第一版（48 端口、单个 ADC 时钟）相比：LUT 5558 → 5653，Bonded IOB 48 → 49（新增 `ad_clk2` 输出）。

综合 0 critical warning 的前提是修正了 `constr/enc_top.xdc` 的缺陷（见第 8 节）。 最早的版本报 **48 critical warnings**，其中 `NSTD-1`（Unspecified I/O Standard）与 `UCIO-1`（Unconstrained Logical Port）各覆盖 13 个端口： `biss_sl, biss_slo, irq_n, key_n, led[3:0], spi_miso, spi_mosi, spi_sck, spi_ss_n, uart_tx` —— 它们既无 `PACKAGE_PIN` 也无 `IOSTANDARD`。补 `IOSTANDARD` 后 `NSTD-1` 消失， 补真实 `PACKAGE_PIN` 后 `UCIO-1` 也归零（48 → 1 → 0）。

---

8. `constr/enc_top.xdc` 修复

#	缺陷	修复
1	13 个端口无 `IOSTANDARD`（NSTD-1 ×13）	补 `set_property IOSTANDARD LVCMOS33`（`key_n`、`{biss_sl biss_slo}`、`{spi_sck spi_ss_n spi_mosi spi_miso}`、`{led[*]}`、`{uart_tx irq_n}`）
2	`key_n`（真实端口）完全无约束；注释里的 `rst_key_n` **不是** `enc_top` 的端口	更正注释 + 补 `IOSTANDARD` 与 `set_false_path`
3	`set_false_path ... uart_rx` 引用了**不存在**的端口（设计只有 `uart_tx`）	删除 `uart_rx`（综合时原本会报 `WARNING: [Vivado 12-584] No ports matched 'uart_rx'`）

#	缺陷	修复
4	ADC 输入延迟绑在反相生成时钟 `adc_clk` 上，跨时钟需求被算成只有 10 ns	改绑 **`sys_clk`**，`max=7.000` / `min=-1.000`（依据见下）
5	OTR 输入延迟 `max 8.000`，比数据总线多 1 ns 且无器件依据	与数据总线统一为 `max 7.000`

第 4 条是本次实现阶段最重要的发现。 第一次 `route_design` 实测：

```
Inter Clock Table
From Clock    To Clock    WNS(ns)   TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints
adc_clk       sys_clk     -4.073    -80.246                     26                   26
```

26 条 `adc_clk → sys_clk` 路径全部不收敛。工具给的 Requirement 是 `10.000ns (sys_clk rise@20.000ns - adc_clk rise@10.000ns)` —— 因为 `adc_clk` 是用 `-invert` 声明的反相时钟，工具认为它的上升沿落在 clk50 下降沿之后。 但物理真相（XDC 自己的第 86-91 行推导）是：ADC 输出寄存器在 `ad_clk` 上升沿更新， 而该沿与采样它的 clk50 上升沿在引脚上是**同一物理时刻**；数据在 clk50 上升沿后 约 `tPD=2..6 ns + 走线 1 ns` 才有效，捕获发生在**下一个** clk50 上升沿。 真实建立窗口是 20 ns 周期里的一大段，**绝不是 10 ns。**

改绑 `sys_clk` 后，工具得出的需求是 `20 - 7 = 13 ns`，与物理一致。

8.2 `PACKAGE_PIN` 的定案依据（当前 50/50；本节记录的是加桥前的 49/49）

8.2.1 ★ 一个必须先纠正的陷阱：厂商表的 "J6/J7/J11" 不是本板的排针

用户提供的厂商文件里，`AD9238_carrier_pin_map.xls` 工作表 7 的表头原文是：

```
双AD9226/AD9238对应管脚    XC7A35V2.0T扩展口J6对应管脚
双AD9226/AD9238对应管脚    XC7A35V2.0T扩展口J7对应管脚
双AD9226/AD9238对应管脚    XC7A35V2.0T扩展口J11对应管脚
```

`XC7A35V2.0T` 不是达芬奇-lite。 这三个口是**另一块板**的扩展口。 本板上：`J7` 是 `RGB_TFTLCD`（RGB-LCD + 触摸排座，2×20，全 bank 34/35）， 而且**根本没有 `J11` 连接器**（FGG484 上的 `J11` 是 `VCCBRAM` 球）。

三列各自的 bank 归属（用 Vivado 的 `get_package_pins` 逐个查证）：

厂商列	bank 归属	在本板上对应什么
厂商 `J6` 列	**38/38 全在 bank 14**	以太网 / HDMI / IIC / TF 卡 / LED —— **未引出到任何排针**
厂商 `J7` 列	**39/39 全在 bank 16**	← **只有这一列落在真实排针上**，即本板的 `J6`
厂商 `J11` 列	**39/39 全在 bank 34**	RGB-LCD 排座 `J7`，含 `V4` =sys_clk

> ⚠️⚠️ **但是：即使选对了列，厂商列的「针号」与本板 `J6` 排针的「针号」也一个都对不上。** 例如厂商 `J7` 列把 `AD2CLK` 放在第 9 针的 `B13`，而本板 `J6` 第 9 针是 `F16`。 **逐位比对 28 个信号：0/28 匹配。** 原因：厂商列是 `XC7A35V2.0T` 那块板的 `J7` 口 pinout，PCB 不同、球号分配不同。 ⇒ **绝对不能把厂商列直接抄进 XDC。**

另一处独立矛盾（说明"直插直用"的确切含义）：模块说明书 p9 §6 自己写 **pin 6/7/8 = 不用**，而厂商 `J7` 列在这三个位置放的是 `V22`(sd_mosi) / `W22`(iic_scl) / `W21`(uart_txd)。⇒ 商家的"完美适配、直插直用"**只对厂商自己的 XC7A35V2.0T 板成立**， 插到任何第三方板上都不可能是 1:1 对位。 **这一条本身就是本工程最有价值的发现之一：照抄表格会直接烧板子。**

8.2.2 权威针→球表的来源（逐针量取，不靠猜）

因为厂商列不可用，本板的针→球映射是**从 `_refs/vendor/Davinci_Lite_1V0.pdf` 原理图 sheet 13 `13_DEVICE_2` 的矢量几何逐针量取**得到的 （符号框坐标 + 引线端点 + 球标签位置，用自写的纯 stdlib `tools/pdf_text.py --geom`）：

- **`J6`** = `Header_2*26`，52 针，bank 16，电源轨 `VCCIO_B16`； `J6-1 = +5V`、`J6-3 = VCCIO_B16`。
- **`J5`** = `Header_2*26`，52 针，bank 15，电源轨 `VCCIO_B15`； `J5-1 = +5V`、`J5-3 = VCCIO_B15`；48 个信号球**全部** bank 15 且未被板载外设占用。

该表由**两个独立来源互证**： 1. 模块说明书 `_refs/vendor/HSTAD9238_module_manual.pdf` p9 §6 的 J6 针表； 2. 载板原理图 `_refs/vendor/AD9238_carrier_schematic.pdf` 的 Altium 文本层 netlist （机器可读 token：`PI<refdes><pin>` / `NL<netname>`；样例 `PIJ6026 PIU1063 → NLAD1CLK`、`PIJ609 PIU1018 → NLAD2CLK`）。

最终校核脚本 **`tools/pinout_verify.py`**（纯 stdlib，内嵌 J6/J5 针表 + 模块网络名 + 板载占用表） 输出 7 项全 PASS：

```
[A] 引脚存在于 xc7a35tfgg484 封装          → 49 / 49
[B] bank 正确性                            → AD9238 组 28 个全 Bank 16；
                                             输出组 16 个全 Bank 15；板载 5 个 bank 正确
[C] XDC 中重复分配的引脚                   → ✓ 无重复（49 个端口 49 个球）
[D] 扩展口引脚是否撞板载外设               → ✓ 0 个命中占用表
[E] 端口完整性                             → RTL 端口 49；已约束 49；未约束 0；全部 LVCMOS33
[F] 球号能否在 J5/J6 针表定位              → ✓ 49 / 49
[G] 载板 28 个信号逐位一致                 → ✓ 28 / 28
RESULT: PASS
```

`tools/pinplan_check.py` 仍然保留，它回答的是另一个问题 （"为什么只能是 bank 16 那一列"，即上面 8.2.1 的三列比对）。

8.2.3 最终接线结果（49 个端口，全部 `LVCMOS33`）

> **★ 2026-10-10 更新**：加入 UART→SPI 桥时新增了 `uart_rx`（**T20**，板载 CH340E 的 RXD）， 端口数 49 → **50**， `tools/pinout_verify.py` 已把 `uart_rx` 加进板载白名单并复跑 **`RESULT: PASS`（`RTL` 端口数 `50`；已约束 `50`；未约束 `0`、`50` 个端口全部有球号 + `IOSTANDARD`、`28/28` 逐个一致）**。 本节下面的逐项数字（49/49）是**加桥前**的记录，保留原文。

模块插 `J6`（bank 16）——28 个 AD9238 信号：

载板丝印	模块针 = J6 针	FPGA 引脚	端口
AD1CLK	26	**B18**	`ad_clk`

载板丝印	模块针 = J6 针	FPGA 引脚	端口
AD1D0…D11	27…38	**F18 E18 C17 D17 A18 A19 C18 C19 F20 F19 B20 A20**	`ad_ch1_d[0..11]`
AD1_OTR	39	**E19**	`ad_ch1_otr`
AD2CLK	9	**F16**	`ad_clk2` **★ 独立第二路时钟**
AD2D0…D11	10…18, 21…23	**E17 C13 B13 E16 D16 A13 A14 D14 D15 C14 C15 B15**	`ad_ch2_d[0..11]`
AD2_OTR	24	**B16**	`ad_ch2_otr`

> ★ **关键事实**：载板上 `AD1CLK`（J6-26 + U1-63 `CLK_A`）与 `AD2CLK`（J6-9 + U1-18 `CLK_B`）是**两根独立网络**（无共用网络、无时钟缓冲器、无板上晶振）。 所以**必须用两个 FPGA 引脚分别驱动**，因此 `rtl/enc_top.v` 新增了第二个 `ODDR`（`u_ad_clk2_oddr`）与端口 `ad_clk2`，XDC 中也新增了对应的 `create_generated_clock -name adc_clk2 … -invert`。 （数据复用模式需要 `MUX_SELECT` = U1 pin 61，它在载板上被固定连接、未引到排针。）

输出侧 16 个端口走 `J5`（bank 15）——因为 `J6` 被载板占满 1…40 针， 只剩 6/7/8、19/20、25、40 七个"不用"位且其球号无法量出：

FPGA 引脚	J5 针	端口
H20	5	`a_out`
G20	6	`b_out`
M21	7	`z_out`
L21	8	`u_out`
N22	9	`v_out`
M22	10	`w_out`
N20	11	`biss_ma`
M20	12	`biss_sl`
L19	13	`biss_slo`
L20	14	`spi_sck`
J22	15	`spi_ss_n`
H22	16	`spi_mosi`
J20	17	`spi_miso`
J21	18	`led[2]`（本板无灯，引出作状态探针）
K21	19	`led[3]`（同上）
K22	20	`irq_n`

板载专用引脚（不占用排针）：

端口	FPGA 引脚	位置	Bank
`sys_clk`	**V4**	板载 50 MHz 晶振	34
`key_n`	**W9**	板载 KEY0	34
`led[0]`	**AB21**	板载 LED	14
`led[1]`	**AB22**	板载 LED	14
`uart_tx`	**W21**	板载 USB 串口	14

> **★ 上电前必须做的一步**：bank 16 的 VCCIO 网络是 `VCCIO_B16`， **不是固定 3.3 V**；它由三位跳线 `J10`（`MINI_BOOT_3*2`，丝印 `OFF` / `ON`） 从可调 LDO `U22`（`SPX3819M5-L-TR`， `VIN=+5V`）选择， 丝印图例：`vout = 1.235 × (1 + R1/R2)`， `3.3V → R1 = 79K`（`R139`）。 bank 15 同构（跳线 `J9`、网络 `VCCIO_B15`）。 ⇒ **`LVCMOS33` 只有把 `J9` 与 `J10` 都实际拨到 3.3 V 才成立**， 否则会触发 "Bank VCCO 与 IOSTANDARD 不符" 的 DRC，或直接损坏 IO。 完整的针号→丝印→引脚对照表与分级上电检查清单见 `docs/04_wiring_checklist.md`。

---

9. 实现结果（`tools/impl_check.tcl`，含真实 place & route 时序）

```
Synthesis finished with 0 errors, 0 critical warnings and 46 warnings
place_design : Post Placement Timing Summary WNS=1.206
route_design : Router Completed Successfully
               Number of Failed Nets = 0, Unrouted Nets = 0, Node Overlaps = 0
               Router Completed Successfully / 10 Infos, 0 Warnings,
               0 Critical Warnings and 0 Errors
write_bitstream : INFO: [Vivado 12-1842] Bitgen Completed Successfully.
                  Bitstream compression saved 11603072 bits
                  （= 省掉的位数：未压缩 17,537,696 bit − 11,603,072 = 5,934,624 bit 压缩后）
                  Writing bitstream C:/Users/zhaoy/Desktop/FPGA/_synth_work/enc_top.bit
```

9.1 时序签核（`report_timing_summary`，Fully Routed）

> **★ 本节是"输出侧约束补齐之前"的历史基线（WNS +0.544 ns）。** 2026-10-10 给 14 个输出补上 `set_output_delay` 后重跑实现，那一轮的签核数字是 **WNS +0.512 ns / 0 失败端点 / 位流 731,612 B**（§12.2）； 修掉 §1.18（幂等标定）后再次重跑，那一轮的签核数字是 WNS +0.923 ns / 9631 端点 / 位流 962,600 B（§12.3，**历史**）。本表保留用于对比。

指标	数值
WNS	**+0.544 ns**（无违例）
TNS	**0.000 ns**（0 条失败端点 / 9474 条）
WHS	**+0.089 ns**（无违例）
THS	**0.000 ns**（0 条失败端点 / 9474 条）

指标	数值
WPWS / TPWS	9.146 ns / 0.000 ns（0 条失败 / 3467 条）
结论行	`All user specified timing constraints are met.`

分时钟（`report_clocks`/Clock Summary）：

Clock	波形	周期	频率	WNS	WHS	失败端点/总数
`sys_clk`（板载 50 MHz）	`{0.000 10.000}`	20.000 ns	50.000 MHz	+0.544	+0.079	0 / 9474
`adc_clk`（生成时钟，`-invert`）	`{10.000 20.000}`	20.000 ns	50.000 MHz	+6.965	+0.965	0 / 1
`adc_clk2`（第二路生成时钟）	`{10.000 20.000}`	20.000 ns	50.000 MHz	+6.965	+0.965	0 / 1

跨时钟只剩两条，且都是**正裕量**：

From	To	WNS	WHS	失败端点/总数
`sys_clk`	`adc_clk`	+6.697	+0.732	0 / 1
`sys_clk`	`adc_clk2`	+6.697	+0.733	0 / 1

> `adc_clk → sys_clk` **这一行从第一次运行的** `WNS=-4.073 / 26` 条失败 **变成了 "不存在"** —— 因为修正后的输入约束绑到 `sys_clk`，ADC 捕获通路不再是一条 跨时钟路径，而是同域路径（含在 `sys_clk` 的 9474 条里，正裕量 +0.544 ns）。 这是第 8 节第 4 条修复的直接效果。
>
> 与第一版（48 端口 / 单个 ADC 时钟）相比 WNS 从 +1.445 → +0.944 → **+0.544 ns**： 一是新增了第二个 `ODDR`（`u_ad_clk2_oddr`）并把 16 个输出端口重排到 `J5`， 二是修复 `abz_gen` 累加器缺陷时把差路径加宽到 13 位有符号、`acc_th` 改为有符号， 临界路径随之改变。**+0.544 ns ≈ 3.4% 周期余量仍然充足**，但要注意这是 单点余量 —— 若后续提高 `sys_clk` 频率需重新签核。

9.2 布线后 DRC（`impl_drc.rpt`）

```
Checks found: 43
| Rule   | Severity | Description            | Checks |
| DPIP-1 | Warning  | Input pipelining       | 11     |
| DPOP-1 | Warning  | PREG Output pipelining | 16     |
| DPOP-2 | Warning  | MREG Output pipelining | 16     |
```

- **0 errors、0 critical warnings**。此前唯一的 `UCIO-1`（Unconstrained Logical Port， 覆盖 6 个端口）在 **50/50** 引脚分配完成后**已消失**（2026-10-10 加 UART→SPI 桥后端口由 49 变 50）。
- `DPIP-1` / `DPOP-1` / `DPOP-2` 共 43 条：都是 DSP48E1 的流水线建议 （输入/PREG/MREG 寄存器），**不是缺陷**；当前 20 ns 周期下裕量 +0.544 ns 已经足够， 若将来提升到 100 MHz 再考虑加流水。

9.3 能否生成 bitstream

> ★ 本节是**输出侧约束补齐之前**的历史基线（WNS +0.544 ns / 741,836 B）。 **（2026-10-10 历史签核数字见 §12.4）** （WNS −3.352 ns、位流 804,780 B @ 2026-10-10 16:05:29）； **★（2026-10-12 v12）现行时序签核：WNS +0.959 ns / TNS 0.000 / 0 个失败端点（11,275 端点），** `All user specified timing constraints are met.` **，位流** `enc_top.bit` **1,042,208 B @2026-10-12 23:59:27（ILA 版 WNS +0.332 / 0 失败 / 20,165 端点、位流 1,428,064 B @2026-10-13 07:12:11）； v11（历史，已归档** `_synth_work/enc_top_v11prod.bit` **）：WNS +0.821 / 0 失败 / 11,310 端点、位流 1,086,600 B @21:15:37**； P0-2 那一轮见 §12.2（+0.512 ns / 731,612 B）；ILA 版见 §12.1。

步骤	结果
综合	✅ 0 errors、0 critical warnings
布局布线	✅ 0 failed nets、0 unrouted nets、0 node overlaps
时序	✅ **当时全约束满足**（当时 WNS +0.544 ns；08:40 轮 +0.923 ns；**2026-10-10 16:05 轮 v9 历史为 −3.352 ns，见 §12.4**； **（2026-10-12 v12）现行为 WNS +0.959 ns / 0 个失败端点 / 11,275 端点，结论行** `All user specified timing constraints are met.` **（v11 历史：+0.821 ns / 11,310 端点）**）
DRC	✅ 43 checks，0 errors、0 critical warnings
bitstream	✅ **成功**：`_synth_work/enc_top.bit`（当时 **741,836 B**；08:40 轮 962,600 B；**2026-10-10 16:05 轮 v9 历史为 804,780 B，见 §12.4**； **（2026-10-12 v12）现行为 1,042,208 B @2026-10-12 23:59:27（v11 历史：1,086,600 B @21:15:37，已归档** `_synth_work/enc_top_v11prod.bit` **）**）
ILA 调试版 bitstream	✅ **成功**：`_synth_work/enc_top_ila.bit`（**现行 v12 版为 1,428,064 B** @2026-10-13 07:12:11）+ `enc_top_ila.ltx` **71,337 B** @07:12:00（GOLD 许可证；含探针位序修复 + `0x0405[0]` OTR 软清位 + UART→SPI 桥）；**v11 历史：1,458,744 B @2026-10-12 21:29:54（已归档** `_synth_work/enc_top_ila_v11.bit` **）；2026-10-10 16:12 轮：1,445,812 B @16:12:01 + ltx 67,006 B @16:11:50（见 §12.1，上一版 15:14 的 1,435,240 B 已作废）**）

即：**从 RTL 仿真、综合、实现、引脚约束到 bitstream 生成，整条链路已在本机打通。**

---

11. 已知缺口与未解决事项（如实写明）

#	事项	状态
1	~~`PACKAGE_PIN` 全部是 TODO~~	✅ **已关闭**：**当前 50/50** 端口已绑定真实引脚（见 8.2），`tools/pinout_verify.py` 与 `tools/pinplan_check.py` 都 `RESULT: PASS`，bitstream 已成功生成

#	事项	状态
2	~~UVW 迟滞 `cfg_hyst_uvw[4:0]` **没有接线** ~~	✅ **已接通（2026-10）**，见 §4.14。`0x0104 != 0` 时该值直接作为角度 LSB 死区（1 LSB = 0.0054932° 电角度，上限 31 LSB = 0.170°）；`0x0104 == 0x00` 时回退到 compilation 参数 `UVW_HYST_SHIFT`（默认 6 → 1024 LSB = 5.625°）。★ 原"死区被 `SECT_W = 8192` 粗栅格淹没、只在极窄窗口内生效"的既有局限**已于 2026-10 修掉**（迟滞基准改用真实扇区边界 `ceil(k*2^AW/6)`），见 §4.15
2b	✅ ~~`uvw_gen` 的 `eang*6` 被位宽截断成 `eang*5` ~~	✅ **真实 RTL 缺陷，已修**，见 §4.13。原式 `{eang,2'b00} + {eang,1'b0}` 两操作数各 18 位 ⇒ 结果截断为 `(eang*6) mod 2^18`，`sec_raw` 实际取 `eang*5` 的高 3 位 ⇒ 65536 个角度里 32768 个（50.0%）扇区号偏小 1，边界由 `10923/21846/32768/43691/54614` 变成 `13108/26215/39322/52429`，并多出 1 拍毛刺态。修法 `({3'b0,eang} << 2) + ({3'b0,eang} << 1)`（19 位足够）。`sim/tb_uvw_gen.v` TEST 9 全 65536 点扫描验证通过
3	`rtl/uvw_gen.v:125` 的 `step_f` **从未归一化**到 [−3,+3]	仅在大幅反向跳变（>1 扇区/拍）时判错方向；单调斜坡永不触及。未改（最小改动原则）
4	`rtl/abz_gen.v` 累加器在**满分辨率 div=1、inter=256** 下的进位排空限制	`d_sgn==0` 的样点无法排空残留进位；div≥4 时有余量故不丢。修法 (a) 会与黄金模型的 `d_sgn != 0` 门控分叉，故保留现状并在报告说明

#	事项	状态
5	`model/abz_uvw_model.py` **不是 uvw_gen 的忠实 oracle**	`:342` 的 `sec_of` 用 `((eang*6)>>aw)%6`，RTL 用 `eang_x6[AW+2:AW]`，扇区边界附近不一致；`:390` 仍是旧的连续告警逻辑。CORDIC 的 oracle（`golden_model.py`）是忠实的，已对账
6	`sim/tb_cordic_atan2.v:326` 注释与实际基准表不符	注释写 `k*128`，实际是 15° 起步 +128 步进；不影响判决
7	`sim/tb_pole_counter.v` 的 `pair` task 有跨调用污染	5 个被检查值不受影响，但 `cycle_cnt` / `dir` 监视器会被扰动
8	`rtl/ad9238_capture.v` 头注释假设 `clk_cap` 是 MMCM 180° 相移，实现里直接接 `clk_i`	实现与文档不一致；`rtl/enc_top.v:187` `en` 硬接 `1'b1`

#	事项	状态
9	`_tools/tmp`、`_tools/dl` 两个 pip 临时目录带沙箱拒绝 ACL，删不掉	与本工程无关
10	`tb_abz_gen` TEST C 最后一组打印 "div=3" 但实选分频 4	只断言 `z_pulses==1`，两边都过；装饰性
11	★ `sim/tb_enc_top.v` 的"一圈"定义与 A 边沿期望值**与 SPEC 不符**	深度探针（`_sim_work/pt_*.v`）实测证实 **RTL 侧正确**：`cyc_max=64`、每 64 个电周期恰好 1 个 Z、A 边沿率 = SPEC `4·inter/div`（误差 <3%）。旧 TB 把"一圈"写成 `64*640` 拍（应为 64×4096=262144 拍），并把 SPEC 的 `4·inter`（一个电周期、div=1、A+B 合计）当成"每圈只数 A"⇒ **是测试台期望值 bug，不是 RTL 缺陷**。当前回归仍 13/13 PASS，因为相关断言的容差恰好兜住了

#	事项	状态
12	✅ ~~`rtl/abz_gen.v` 累加器会**静默丢 A 边沿** ~~ → **真根因已定位并修复**	✅ **真实 RTL 缺陷，已修并验证。真根因不是"符号门控"，而是有符号/无符号混比**：`acc_th` 原先声明为 `wire [ACCW-1:0]`（**无符号**），而点火判据是 `if (acc_step_sum >= acc_th)`。Verilog 混符号比较的规则是"只要有一边无符号就两边都按无符号比"，而 `acc_step_sum` 是 `wire signed [ACCW-1:0]`。于是累加器为负时（**反向路径必然出现**）`-1` 被读成 `8191`，`8191 >= 1` 成立 ⇒ **反向也被判成"正向点火"**，真正的反向排空分支 `acc_step_sum < ACC_ZERO` 永远轮不到，累加器单调滚向负饱和、A/B 边沿静默丢失，而 `lag_warn` / `lag_fatal` 全程为 0。逐拍 dump 证据（`_sim_work/pt_e3.v`，`abzdir=1 / inter=8 / div=0`，改前）：`i=16 acc_sum=-2 acc_th=1 fire_f=1 fire_b=0 r_acc=-2`，此后每拍 −1 单调下滚。**四处修法**：① `acc_th` 改 `wire signed [ACCW-1:0]`（**关键一行**，两边都变成有符号、比较才按补码语义进行）；② `d_fwd_raw` / `d_rev_raw` / `d_sgn` / `d_abs` / `half_per` / `period` 统一加宽到 **13 位有符号**（原 `wire [10:0] d_sgn` 把反向的 −1 存成 2047，`d_sgn[10]=1` + `~d_sgn+11'd1` 算出 −2047）；③ 删掉一行**遗留的重复** `assign step_val`（`assign step_val = trig_ok ? {{(STEPW-11){d_sgn[10]}}, d_sgn} : STEP_ZERO;` —— 它按 `d_sgn[10]` 做零扩展、把 −1 变成 −2047，而且**覆盖了**上面新写的 generate 符号扩展分支）；④ 补 `step_val == 0` 拍的**残留进位排空**（唯一排空点在主时序 `r_acc`，组合块只出判据、绝不自己扣 `acc_th` —— 错误第一版两处都扣、同一拍排空两遍，回归曾掉到 `PASS 11/13`）。黄金模型 `model/abz_uvw_model.py` 同步（并把 `n_step = inter` 修正为 `4*inter`、把变长 `while` 排空改为与 RTL 一致的单次 `if`）。**验证**：隔离台 `tb_abz_gen` TEST A 的 A/B 边沿数**逐项精确等于** `4*inter/div`（`2/1` 总 8、`8/3` 总 10、`64/1` 总 256、`64/4` 总 64、`256/8` 总 128，偏差全部 ≤±1）；TEST E 方向/极性首次真正生效（正常=+249、`apol=1 → -249`、`bpol=1 → -249`、`both=1 → +249`、`abzdir=1 → -249`、`apol+abzdir=1 → +249`）；集成台 `tb_enc_top` 一圈 **A 沿 4096 = SPEC 精确值**、Z 1 个、`max_dev=5 LSB16`、`TEST PASSED (39 checks)`；全量回归 **13/13 PASS**。⇒ **不再需要降低输入速率或回避 `div=1` / `inter=256`**。（原记载的"1.8%~20%"来自只用 4 个采样点/电周期的隔离测试台，属混叠读数，不足为凭）
13	~~`cfg_zwidth` 复位默认值是 0，导致 Z 与 UVW 永久不出~~	✅ **已修**：`rtl/reg_file.v` 的 `r_zero1` 复位值由 `8'h00` 改为 **`8'h04`**。原来 0 会让 `rtl/abz_gen.v` 的 `assign z_pulse_n = (z_width == 4'd0) ? 4'd0 : z_width;` 恒为 0，Z 永不输出，`rtl/uvw_gen.v:232-233` 也把 u/v/w 同时钉死。4 个 AB 边沿步（★ 2026-10-10 更正：不是 50 MHz 拍，1 kHz/inter=256/div=1 时 1 步 = 977 ns ⇒ ≈3.9 µs，见 DEV_STATUS §1.26）对下游计数器足够宽。改后 **13/13 回归仍 PASS**、实现数字不变。⚠️ **2026-10-11 追加发现**：复位值修好≠宽度生效 —— 集成路径上推进逻辑被 `z_window` 门控窗截断，实际只有 2 clk = 40 ns，v8 已修（见 DEV_STATUS §1.44）。**★ 2026-10-11 v9 再修语义**：`z_pulse_n` 改为 **`{1'b0, z_width} + 5'd1`**（按 TW28 手册 `zwidth+1`）⇒ 同一 `z_width = 4` 现在是 **5 步 = 4.88 µs**，且 `z_width = 0` 也出 1 步（不再"无 Z"），见 DEV_STATUS §1.49。**★ 2026-10-11 板上闭环（ILA，v9 位流）**：`z_width = 4` 实测 240 拍 = **5 个边沿步**、`0x0108 = 0x4000` 实测 60 拍 = **1 个边沿步**（见 DEV_STATUS §1.26 板上实证块）
14	★ 调试版 ILA 的两个流程坑（已写进 `tools/ila_debug.tcl` 头注释）	(1) `read_verilog -define` 在非工程模式直接报 `ERROR: [Common 17-69] ... '-define' is only supported when is_compile_unit_mode is enabled on the source fileset` ⇒ 宏只能通过 `synth_design -verilog_define ENC_TOP_ILA` 给。(2) `get_debug_ports` **只枚举已存在的 debug core 的端口**；在 `create_debug_core` 之前调用只会打印 `WARNING: [Vivado 12-750] There are no ChipScope debug cores in this design.` 并返回空表（这一点让第一版脚本误判为"探针没加上"而 `exit 1`）⇒ 顺序必须是 `opt_design → create_debug_core → get_debug_ports → connect_debug_port`，探针网本身用 `get_nets -hier -filter {MARK_DEBUG==1}` 取（实测 252 条）

#	事项	状态
15	**★ `rtl/enc_top.v` 的标定窗口判据接错，窗口永远关不上**	✅ **真实 RTL 缺陷，已修**（2026-10-10）。`calib_ctrl` 的 `.cycle_pulse` 原接 `u_pole` 的 `wrap_pos|wrap_neg`，而标定期间 `loop_filter` 被 freeze（`enc_top.v:395 .freeze(calib_busy)`）⇒ 角度永不跨周期边界、`wcnt` 恒 0、`win_cycles=64` 这条路径**永远走不到**，窗口只能靠 `MAX_SAMP=65536` 截断（实测修复前 `nsamp=65536 / wcnt=0 / truncated=1`）。修法：改用**未被冻结**的 CORDIC 原始角度 `angle_raw` 做整圈折返计数（MSB 翻转 = 半圈，两次 = 一个输入周期）产生 `calib_cycle_pulse`。修复后 `nsamp=17192 / wcnt=65 / truncated=0`。**为什么此前没发现**：模块级 `tb_calib_min` 自己造 `cycle_pulse`，顶层 `tb_enc_top` 从不触发标定 ⇒ 接法没人走过。已在 `sim/tb_enc_top.v` 新增 TEST F（39 → **63 checks**）覆盖整条集成路径
16	**★ 一键标定不可重复（标定看的是自己的输出 + 窗口不是整周期）**	✅ **真实 RTL 缺陷，已修**（2026-10-10）。**两个来源，只修一个不够**：① `calib_ctrl` 累加的是 `sc_correct` 的**输出**（`.in_valid(cor_valid)`、`.s_n(s_cor)`、`.c_n(c_cor)`），而系数在第一次标定后已生效 ⇒ 第二次标定看到的是"已被校正过"的信号，把系数往回掰。改为累加校正前的原始抽取样点 `s_dec/c_dec` 后重跑，**仍然 FAIL（5 errors/64 checks，两次标定之差 `OFS_C=-947`）**。② `S_ACCUM` 从 `start` 那一拍就累加 ⇒ 窗口 = "起始残缺周期 + win_cycles 个整周期"，残缺段长度取决于触发相位（实测 `nsamp=17192 / wcnt=65`，而 64 整周期应为 16384）⇒ 正交性被破坏、偏置/相位估计被幅度泄漏污染。修法：新增 `armed` 对齐逻辑，**先等到第一个输入周期边界再累加**（`ARM_TMO_MAX=1 s` 超时保护，输入不转时 `truncated=1` 空窗口收尾不挂死），关门判据回到 `(wcnt+1) >= win_n`。★ 踩坑记录：对齐那一拍的样本**要计入**、周期**不计**（`wcnt` 自增加 `armed` 保护），否则分别得到 16383 与 16128。修复后（`tb_enc_top` 72 checks）：`nsamp=16384 wcnt=64`、两次标定六项结果**逐位一致**（差值全 0），`tb_calib_min` 未改动即通过

12. 复现产物清单

路径	说明
`tools/run_sim.ps1`	批量回归脚本（PowerShell；需 UTF-8 **BOM**）
`tools/lint_all.ps1`	28 个 .v 逐文件独立编译体检
`tools/synth_check.tcl`	综合验证（`xc7a35tfgg484-2`）
`tools/impl_check.tcl`	实现验证（opt/place/route + 真实时序 + `write_bitstream`）→ 生产版 `enc_top.bit`

路径	说明
`tools/ila_debug.tcl`	**调试版流程**：同一套 RTL/XDC + `-verilog_define ENC_TOP_ILA`，插 ILA → `enc_top_ila.bit` + `enc_top_ila.ltx`。★ **需 GOLD 许可证**（见上表；未设 `XILINXD_LICENSE_FILE` 时会报 `ERROR: [Vivado 12-29205]` 命中的是 BASIC）。已按 2026.1 API 重写为**命名多端口**：`synth_design` 后收 `MARK_DEBUG=1` 的网（实测 252 条）→ 按基名归组、宽于 32 bit 的切片 ⇒ **34 条探针端口、总位宽 252**；顺序必须 `opt_design → create_debug_core → get_debug_ports → connect_debug_port`；配置属性挂在 **core** 上（`set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila]`），对 `get_debug_ports` 设会报 `12-4442`。★ **沙箱注意**：IP 生成会派生一个带管道的子 vivado 进程，受限沙箱下父进程无限等待 ⇒ 该脚本需单独放宽一次权限；生产版 `impl_check.tcl` 不生成 IP、不需要
`tools/reconcile_cordic.py`	RTL↔黄金模型 CORDIC 三角对账
`tools/pinplan_check.py`	引脚方案校核（三列 bank 比对；`RESULT: PASS`）
`tools/pinout_verify.py`	**引脚终版校核**（A–G 七项：球存在 / bank / 无重复 / 不撞外设 / 端口全覆盖 / 可定位 / 载板 28 信号逐位一致；`RESULT: PASS`）
`tools/pdf_text.py`	纯 stdlib 的 PDF 文本+矢量提取器（含 `/ObjStm`、PNG predictor、CMap 中文、`--geom`）。原理图几何量取与 netlist 提取都靠它
`tools/xls_read.py`	纯 stdlib 的 OLE2+BIFF8 读取器（解析厂商 `.xls` 引脚表）
`docs/04_wiring_checklist.md`	40 针接线表 + 上电检查清单
`sim/logs/*.log`	19 个测试台的完整 xsim 输出（UTF-8 BOM）
`_synth_work/impl_timing.rpt`	实现后真实时序报告

路径	说明
`_synth_work/impl_util.rpt` / `impl_drc.rpt`	实现后资源与 DRC
`_synth_work/enc_top.bit`	**可烧写的生产版 bitstream。（2026-10-12 v12）现行为 1,042,208 B @2026-10-12 23:59:27（SHA256 `D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725`）★ 上板请用这一版。v11 历史（已归档 `_synth_work/enc_top_v11prod.bit`）**：1,086,600 B @2026-10-12 21:15:37。**2026-10-10 v9 历史**：**804,780 B**，2026-10-10 16:05:29，`COMPRESS=TRUE`（见 §12.4）；12:08 的 1,219,152 B 与 08:40 的 962,600 B 都已作废（前者在 `_synth_work/signoff_adc_honest/`、后者在 `signoff_0840/`）
`_synth_work/enc_top_ila.bit`	**可烧写的调试版（含 ILA）bitstream。（2026-10-12 v12）现行为 1,428,064 B @2026-10-13 07:12:11（SHA256 `5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`，时序同判据全绿，WNS +0.332 / 20,165 端点；36 条命名探针 / 269 bit）；v11 历史（已归档 `_synth_work/enc_top_ila_v11.bit`）**：1,458,744 B @2026-10-12 21:29:54。**2026-10-10 16:12 版为历史**：**1,445,812 B**，2026-10-10 16:12:01 —— 含探针位序修复 + `0x0405[0]` OTR 软清位 + UART→SPI 桥（上一版 1,435,240 B @15:14:21 已移到 `_synth_work/signoff_pre_uart_bridge/`，1,423,056 B @08:56:25 在 `signoff_ila_bitrev_old/`，更早的 1,148,904 B @07:44 与 1,473,128 B @10-09 23:31 均已作废）
`_synth_work/enc_top_ila.ltx`	**ILA 调试探针描述文件**（**v12 现行 71,337 B @2026-10-13 07:12:00**；2026-10-10 16:12 轮为 67,006 B @16:11:50，属历史；Vivado Hardware Manager 必须与 `.bit` 配套加载）
`_synth_work/impl_ila_{timing,util,drc}.rpt`	调试版实现的时序 / 资源 / DRC 报告
`_synth_work/ila_probe_list.txt`	**v12 现行：36 条命名探针 / 269 bit** ↔ 信号名 ↔ 位宽对照表（2026-10-10 16:12 轮曾为 34 条 / 252 bit，属历史；表头为 ASCII；编码坑的复盘见 `docs/DEV_STATUS.md` §1.7）

12.1 调试版（ILA）实测结果 —— 已跑通（2026-10-10 16:12 用当前 RTL 重建）

`tools/ila_debug.tcl` 在 GOLD 许可证 + 一次放宽沙箱权限下**完整跑通**，全部报告如下。

项	结果
`synth_design -verilog_define ENC_TOP_ILA`	成功；`MARK_DEBUG=1` 的网实测 **252 条**（日志 `ILA: nets with MARK_DEBUG = 252`）
`create_debug_core u_ila_0 ila` + 连接探针	成功；`ILA: distinct probe bases = 34` ⇒ **34 条命名探针端口、总位宽 252 bit**（`C_DATA_DEPTH 1024`、`C_INPUT_PIPE_STAGES 1`）；`connect_debug_port` 的 net 列表按 **LSB→MSB** 传入（§1.20 位序修复）
采样时钟	`u_bufg_clk/O`（即 `rtl/enc_top.v` 的 `clk_i`；窗口 = 1024 / 50 MHz = **20.48 µs**）
`route_design`	`Router Completed Successfully` → `route_design completed successfully`；`0 Critical Warnings and 0 Errors`

项	结果
时序（Fully Routed）	★ **（2026-10-12 v12：已收敛）** `_synth_work/impl_ila_timing.rpt` @2026-10-13 07:11:57 结论行 **`All user specified timing constraints are met.`**（WNS **+0.332** / TNS 0.000 / **0 失败 / 20,165 端点**、WHS +0.050 / 0 / 20,149、WPWS 8.870 / 0 / 9,858），产出 `_synth_work/enc_top_ila.bit` **1,428,064 B @2026-10-13 07:12:11**（SHA256 `5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`）。**v11 历史：`enc_top_ila.bit` 1,458,744 B @2026-10-12 21:29:54（route 估计 WNS 0.565 / WHS 0.047，已归档 `_synth_work/enc_top_ila_v11.bit`）。以下为 2026-10-10 v9 历史轮次**：（原「2026-10-10 16:12 重建版」，含 UART→SPI 桥 + 探针位序修复 + OTR 软清位 + 诚实 ADC 输入约束）：**`Timing constraints are not met.`**（`impl_ila_timing.rpt:144`），最差路径 `ad_ch1_d[10] → u_capture/ch1_meta_reg[10]/D`（`Input Delay 24.000`、`Data Path 0.881` = logic 0.472 / route 0.409、`Logic Levels 1 (IBUF)`）；整体 **`WNS -3.329 / TNS -84.745 / 26` 个失败端点 / `19161` 端点**，`WHS +0.050 / THS 0.000 / 0 失败 / 19145 端点`，`WPWS 8.870 / 0 失败 / 9422 端点`；`sys_clk` 域 WNS **−3.329 ns**（26 失败 / 18010 端点，**全部是 ADC 输入端口**）、WHS +0.050 / 0 失败 / 18010、WPWS 8.870 / 0 / 8939；`adc_clk` / `adc_clk2` 各 +6.965 / WHS +0.965（各 1 端点），`sys_clk→adc_clk` 跨域 +6.697 / WHS +0.732、`sys_clk→adc_clk2` +6.697 / WHS +0.733（各 1 端点）；`dbg_hub/.../TCK` 域 WNS 27.306 / WHS 0.093 / WPWS 15.370（928 端点，WPWS 483）；`sys_clk↔TCK` 跨域 +19.043 / +31.957（各 8 端点）；`async_default`（TCK→TCK）28.946 / 0.397（100 端点）、（sys_clk→sys_clk）17.378 / 0.235（105 端点）。★ 这 26 条负余量**不是新增缺陷**，而是当时输入约束（`-max 24.000 / -min 19.500`，**总输入延迟预算**口径，v9 反相相位值，根因 = 生成时钟 `-invert`）的必然结果，与生产版 12:08 的 −3.271、16:05 的 −3.352 同源；**签核一律以生产版为准。**历史轮次（15:14 的 −3.268/17063、08:56 的 +0.527/18213、10-09 23:31 的 0.698/18004、07:44 的 0.971/18078）**均已作废**（前两版含诚实约束之后/之前的差别，见 `docs/DEV_STATUS.md` §1.19）
DRC	`Checks found: 62`，**0 error / 0 critical**（`DPIP-1` 24 / `DPOP-1` 17 / `DPOP-2` 17 / `PDCN-1569` 3 / `RTSTAT-10` 1，**全部 Warning 且逐条** **`Related violations: <none>`**）
资源增量（对比 §12.4 生产版 16:05）	Slice LUTs **6010** → **8822**（28.89% → **42.41%**，+2812）；Slice Registers **3857** → **8544**（9.27% → 20.54%）；F7 Muxes 155（0.95%）、F8 Muxes 51（0.63%）；Block RAM Tile **0** → **7.5**（0% → 15.00%，即 1024 深 capture RAM）；DSPs **16** → **17**（18.89%）；Bonded IOB **50**（不变，20.00%）；BUFGCTRL **1** → **2**（6.25%）；`BSCANE2` 1 个、黑盒实例 `u_ila` 与 `dbg_hub` 各 1
bitstream	`Bitstream compression saved 5971264 bits`；`17,537,696 − 5,971,264 = 11,566,432 bit = 1,445,804 B`，实测文件 **1,445,812 B**（同样的 **+8 B** 头部差，逐字节自洽）

项	结果
产出	`_synth_work/enc_top_ila.bit`（**1,445,812 B @ 2026-10-10 16:12:01**）+ `_synth_work/enc_top_ila.ltx`（**67,006 B @ 16:11:50**，探针定义 34 条 / 252 bit 未变故大小不变）；报告 `impl_ila_timing.rpt` @16:11:48 / `impl_ila_util.rpt` @16:11:48 / `impl_ila_drc.rpt` @16:11:50 / `ila_probe_list.txt` @16:10:41；**加桥前**的整套 ILA 产物（`enc_top_ila.bit` 1,435,240 B @15:14:21 等）已备份到 `_synth_work/signoff_pre_uart_bridge/`，更早的位序反转版（1,423,056 B @08:56:25）在 `_synth_work/signoff_ila_bitrev_old/`

⇒ 结论：**★ （2026-10-12 v12：已收敛）生产版 `sys_clk` 域 WNS = +0.959 ns / TNS 0.000 / 0 个失败端点（11,275 端点）**，结论行 `All user specified timing constraints are met.`；**ILA 版同判据行（WNS +0.332 / 0 失败 / 20,165 端点）。v11 历史：WNS +0.821 / 0 失败 / 11,310 端点、ILA 版 21:29:41 同判据行（已归档）。以下为 2026-10-10 v9 历史**：生产版 08:40 版是 **+0.923 ns**（诚实约束前）、12:08 诚实版是 **−3.271 ns**（§12.3）、16:05 版是 **−3.352 ns**（§12.4，含桥）、ILA 16:12 版是 **−3.329 ns**（26 个失败端点，全部 ADC 输入端口，根因 = 生成时钟 `-invert` 相位不匹配）； **★ 签核一律以生产版的 `_synth_work/impl_timing.rpt` 为准**，ILA 版只作调试用途。 上板时先在 Vivado Hardware Manager 里 Program Device 选 `enc_top_ila.bit`、**同一个对话框的 Debug probes file 填同目录的 `enc_top_ila.ltx`**， 再把 34 条探针拖到波形窗；触发布置建议用 `dbg_z_window` 上升沿（⚠️ **不要用 `dbg_adc_valid`**：它在非复位期间恒为 1，`rtl/ad9238_capture.v:163`）。

> **为什么必须重建（2026-10-10 16:12 这一版）**：更早的 ILA 位流都已被取代 —— 10-09 23:31 那版（1,473,128 B）**早于三处功能修复** —— `rtl/calib_ctrl.v`（10-10 00:32，标定窗口 off-by-one）、`rtl/uart_scope.v`（10-10 01:01，组帧 FSM 连发 `tx_start`）、`rtl/enc_top.v`（10-10 07:19， 标定窗口脉冲接错，见 §11 第 15 行）；07:44 那一版又早于 08:37 的**幂等标定修复** （`rtl/enc_top.v` 改累加校正前信号、`rtl/calib_ctrl.v` 新增 `armed` 对齐，见 §11 第 16 行）； 08:56 那一版虽然与**当时**的 RTL 一致，却**早于两处修复** —— 14:51 的 ILA 探针位序修复 （`tools/ila_debug.tcl:137`）、14:59 的 `0x0405[0]` OTR 软清位（`rtl/reg_file.v` / `rtl/enc_top.v`）； 15:14 那一版又**早于 UART→SPI 桥**（`rtl/uart_rx.v` / `rtl/uart_spi_bridge.v` / `rtl/uart_scope.v` 的 `hold` + `tx_idle` / `rtl/enc_top.v` 的二选一，15:xx–16:0x，见 `docs/DEV_STATUS.md` §1.22） ⇒ 只有 **16:12 这一版**与当前 RTL + 工具链完全一致（同一份 RTL、同一份 XDC、`tools/ila_debug.tcl`， GOLD 许可证 + 放宽一次沙箱），探针集不变（252 条 `MARK_DEBUG` 网 / 34 条端口 / 252 bit）。 ★ 上板核验判据：下载这一版后，波形窗里 `dbg_pole_pairs_eff`（probe20）必须显示 **4**（13:39 首次上板用的 08:56 版显示 8，= 位序反转）；并可用 SPI 写 `0x0405 = 0x0001` 清 ADC sticky OTR（`led[0]` / `irq_n` 随之恢复，见 §3 第 9 条与 `docs/DEV_STATUS.md` §1.21）； 还可用板载 USB 串口发 4 个字节验证 UART→SPI 桥（`python tools/uart_spi.py read 0x05F0`，§1.22）。

12.2 输出侧约束（2026-10-10，P0-2）：14 个输出首次纳入签核（已被 §12.3 取代，保留为历史轮次）

此前的缺口：全设计 16 个输出里，**只有 `ad_clk` / `ad_clk2` 有输出侧约束** （`constr/enc_top.xdc:147-150`，相对生成时钟 `adc_clk` / `adc_clk2`，因为它们是喂给 外部 AD9238 的采样时钟）。其余 **14 个输出**（`a_out` / `b_out` / `z_out` / `u_out` / `v_out` / `w_out` / `biss_slo` / `spi_miso` / `led[3:0]` / `uart_tx` / `irq_n`）**没有任何 `set_output_delay`**， 全仓 `set_drive` / `set_slew` 命中数为 **0**。未约束的输出不会报错、只会被工具忽略 —— 这正是它长期没被发现的原因。

已写入 `constr/enc_top.xdc` 第 10 节（按"边沿是不是信息"分两类）：

类别	端口	`set_output_delay -clock sys_clk`	驱动 / 压摆率	理由
位置类	`a_out` `b_out` `z_out` `u_out` `v_out` `w_out` `biss_slo` `spi_miso`	`-max 6.000` / `-min 0.500`（发射沿后 ≈14 ns 内稳定）	`SLEW FAST` + `DRIVE 8`（默认 12 mA → 8 mA）	边沿抖动即位置误差，必须保 FAST；只降 di/dt/振铃
指示类	`led[3:0]` `uart_tx` `irq_n`	`-max 3.000` / `-min 0.500`（≈17 ns 内稳定）	`SLEW SLOW` + `DRIVE 4`	对端自行取样，与位置信息无关
不动	`ad_clk` `ad_clk2`（及 ADC 数据 ODDR）	保持第 7 节原值	不动	其边沿相位关系是 ADC 捕获余量分析（§6/§9）的输入，单独改一路会改相对相位

★ 一个容易搞反的 Vivado 语义（第一版就是在这里翻车的）： 对输出端口，`required_time = 下一个发射沿 (20 ns) − set_output_delay − uncertainty`， 即 `-max` **越大、留给 FPGA 内部的时间越少。**

输出预算的三次迭代（含一次真实失败）：

迭代	预算	WNS	失败端点	位流	结论
①	`-max 12.000` / `-min 1.000`	**−3.736 ns**	**14**	923,944 B	**不满足**：最差路径 `u_amp/amp_low_reg/C → led[1]` 的端口数据 11.701 ns 才稳定，12.000 在数学上不可满足
②	`-max 6.000` / `-min 0.500`（全部）	+0.097 ns	0	909,736 B	全约束满足，但全局最差变成**状态灯** `u_capture/otr_reg/C → led[0]`（OBUF+布线 4.667 ns）⇒ 余量过脆
③ **交付**	位置类 `6.000` / 指示类 `3.000`	**+0.512 ns**	**0**	**731,612 B**	最差路径回到**内部**逻辑 ⇒ 输出约束不再是瓶颈

交付版（迭代③）实现签核数字：

指标	数值
bitstream	`_synth_work/enc_top.bit` **731,612 B**，时间戳 **2026-10-10 07:31:54**
整体 WNS / TNS / 失败端点	**+0.512 ns** / 0.000 / **0**（9487 端点 = **全设计汇总**；Intra Clock Table 的 `sys_clk` 为 9485 = 9487 − `adc_clk` 1 − `adc_clk2` 1，见 `docs/DEV_STATUS.md` §1.8）
整体 WHS / THS / 失败端点	**+0.075 ns** / 0.000 / **0**（9487 端点，同上）
WPWS / 失败端点	9.146 ns / **0**（3484 端点）
全局最差路径	**内部路径** `u_regs/r_inter0_reg[3]/C → u_abz/r_hold_reg[0]/R`，Data Path Delay 18.977 ns（logic 9.535 / route 9.442），Slack +0.512
结论行	`All user specified timing constraints are met.`
DRC	`Checks found: 43`（`DPIP-1` ×11 / `DPOP-1` ×16 / `DPOP-2` ×16，全 Warning），**0 error / 0 critical**
布线	8767 nets fully routed，routing errors **0**
资源	LUT 5656（27.19%）/ FF 3476（8.36%）/ DSP 16 / Block RAM 0 / Bonded IOB 49 / BUFGCTRL 1

指标	数值
压缩	`saved 11684864 bits` ⇒ 5,852,832 bit = 731,604 B，实测 731,612 B（+8 B 头部，与既有口径一致）

⇒ 结论：新增约束把输出端口纳入 setup 检查后，WNS 由 §9.1 的 +0.544 变为 **+0.512 ns**（这 0.03 ns 的差别正是"输出裕量第一次被计入"的代价，而不是设计变差），且**全局最差路径已回到内部逻辑。★ 时序签核以本文件开头（2026-10-12 v12）前置说明与 `_synth_work/impl_timing.rpt` 为准**（§12.4 是 2026-10-10 历史轮次，v11 已归档为 `_synth_work/enc_top_v11prod.bit`），§9.1 的 +0.544 是约束补齐前的历史基线。

仍未做（硬件侧，如实登记）：只做了约束 + 驱动/压摆率，**没有加源端串阻。**
`docs/04_wiring_checklist.md:286` 与 `docs/06_board_test_guide.md:20` 要求 J5 排线 ≤5 cm、必要时串 22~33 Ω；`SLEW FAST` + `DRIVE 8 mA` 只降低 di/dt，真正的振铃解法仍是串阻。上板实测波形后可再收紧这两个预算并重跑一次签核。

12.3 ★★ 生产版签核（2026-10-10 08:40，§1.18 幂等标定修复后重跑）（已被 §12.4 取代，保留为历史轮次）

> **★ 口径补充（2026-10-10 12:08 之后，见 `docs/DEV_STATUS.md` §1.19）**：ADC 输入约束后来按 **真实模型**重写（`set_input_delay -clock sys_clk -max 24.000 / -min 19.500`），重跑得到 **WNS −3.271 ns / 26 个 failing endpoints（全部是 ADC 输入端口）**、`Timing constraints are not met.`。本节这一版（08:40）是**"全部约束满足"的最后一版**，与 12:08 版 **RTL 完全相同、只差输入约束值。** 因此本节所有"全约束满足"结论**只在明确写出"按 08:40 的约束集"时成立**；引用时请一并引用 §1.19，说明 ADC 输入路径的真实余量状态（这是待上板实测确认的边界项）。

为什么必须再重跑：§1.18 修的是**功能**——`rtl/enc_top.v` 的 `calib_ctrl` 改为累加 **校正前**的 `s_dec/c_dec`（`.in_valid(dec_valid)`）、`rtl/calib_ctrl.v` 新增 `armed` / `arm_tmo` / `ARM_TMO_MAX=26'd50_000_000` 对齐逻辑（窗口对齐到输入周期边界）。按"位流必须晚于全部 `rtl/*.v`"的判据，位流必须重生成。

指标	数值（08:40 历史轮次，现行见开头 v12 前置说明；§12.4 亦为 2026-10-10 历史）
bitstream	`_synth_work/enc_top.bit` **962,600 B**，时间戳 **2026-10-10 08:40:17**
整体 WNS / TNS / 失败端点	**+0.923 ns** / 0.000 / **0**（**9631** 端点 = 全设计汇总；Intra Clock Table 的 `sys_clk` 为 **9629** = 9631 − `adc_clk` 1 − `adc_clk2` 1）
整体 WHS / THS / 失败端点	**+0.066 ns** / 0.000 / **0**（9631 端点）
WPWS / 失败端点	**9.500 ns** / **0**（3534 端点）
全局最差路径	**内部路径** `u_regs/r_inter0_reg[3]/C → u_abz/r_hold_reg[10]/R`，Data Path Delay **18.634 ns**（logic 9.802 / route 8.832），27 级逻辑（CARRY4×11、DSP48E1×1…），`Slack (MET) +0.923`
结论行	`All user specified timing constraints are met.`
DRC	`Checks found: 43`（`DPIP-1`×11 / `DPOP-1`×16 / `DPOP-2`×16，全 Warning），**0 error / 0 critical**

指标	数值（08:40 历史轮次，现行见开头 v12 前置说明；§12.4 亦为 2026-10-10 历史）
布线	**8904** nets fully routed，routing errors **0**
资源	LUT **5763（27.71%）** / FF **3528（8.48%）** / DSP 16（17.78%）/ Block RAM 0 / Bonded IOB 49 / BUFGCTRL 1 / BSCANE2 0
压缩	`saved 9836960 bits` ⇒ 17,537,696 − 9,836,960 = 7,700,736 bit = **962,592 B**，实测 **962,600 B**（+8 B 头部，与 §12.2/§9.1 口径一致）
位流头	`00 09 0f f0`（与 §12.2 相同）

⇒ 结论：`armed` 对齐逻辑**没有拖累时序**（WNS 由 §12.2 的 +0.512 提升到 **+0.923 ns**， 最差路径仍是内部寄存器到寄存器，输出端口依然不是瓶颈），代价是 **+107 LUT / +52 FF / +230,988 B 位流**（新增 26 bit 超时计数器 + 比较器 + 对齐分支）。 **★ 本节是历史轮次**（08:40，诚实 ADC 输入约束**之前**）：它说明"设计内部逻辑完全能收敛"， 但**不是**当前签核口径。**现行时序签核以 §12.4 与 `_synth_work/impl_timing.rpt` 为准**； §9.1（+0.544，约束补齐前）与 §12.2（+0.512，P0-2 那一轮）同样只是历史基线。 本节的实测同时覆盖了 §12.2 补的 14 条输出侧约束（当时 `impl_timing.rpt` 全约束满足）。

12.4 ★★ 生产版签核（2026-10-10 16:05:29，含 UART→SPI 桥）

> **★（2026-10-12 v12）本节已降级为 2026-10-10 的历史轮次**：现行签核是 v12 —— 生产版 `_synth_work/impl_timing.rpt` @2026-10-12 23:59:16 结论行 **`All user specified timing constraints are met.`**（`sys_clk` **WNS +0.959 / TNS 0.000 / 0 个失败端点 / 11,275 端点**，WHS +0.075、WPWS 9.500）， 位流 `_synth_work/enc_top.bit` **1,042,208 B @2026-10-12 23:59:27**；ILA 版 `_synth_work/impl_ila_timing.rpt` @2026-10-13 07:11:57 同判据行（WNS +0.332 / 0 失败 / 20,165 端点），位流 1,428,064 B @2026-10-13 07:12:11。 **v11（历史，已归档 `_synth_work/enc_top_v11prod.bit` / `_synth_work/enc_top_ila_v11.bit`）为 WNS +0.821 / 11,310 端点、生产位流 1,086,600 B @21:15:37。** 下表 −3.352 等数字均为 **2026-10-10 v9 历史**。

为什么是这一版：08:40（§12.3）之后 RTL 又变了两处 —— 14:59 的 `0x0405[0]` OTR 软清位 （§1.21）与 15:xx–16:0x 的 **UART→SPI 桥**（§1.22，新增 `rtl/uart_rx.v` / `rtl/uart_spi_bridge.v`、 `rtl/uart_scope.v` 的 `hold`/`tx_idle`、 `rtl/enc_top.v` 的二选一与 `uart_rx` 端口、XDC 的 `T20`）。 按"位流必须晚于全部 `rtl/*.v` / `constr/*.xdc`"的判据，必须重跑。桥之前的整套产物已备份到 `_synth_work/signoff_pre_uart_bridge/`（含 12:08 的 1,219,152 B 与 15:14 的 ILA 1,435,240 B）。

指标	数值（2026-10-10 v9 历史签核，生产版 `_synth_work/enc_top.bit`； （2026-10-12 v12）现行签核为 WNS +0.959 / TNS 0.000 / 0 个失败端点 / 11,275 端点、结论行 `All user specified timing constraints are met.`、位流 1,042,208 B @2026-10-12 23:59:27；ILA 版 WNS +0.332 / 0 失败 / 20,165 端点、位流 1,428,064 B @2026-10-13 07:12:11；v11 历史：+0.821 / 11,310 端点、1,086,600 B @21:15:37）
bitstream	`_synth_work/enc_top.bit` **804,780 B**，时间戳 **2026-10-10 16:05:29**
整体 WNS / TNS / 失败端点	**−3.352 ns** / **−85.116** / **26**（**10582** 端点 = 全设计汇总；全部 26 条都是 ADC 输入端口）

指标	**数值（2026-10-10 v9 历史签核，生产版 `_synth_work/enc_top.bit`； （2026-10-12 v12）现行签核为 WNS +0.959 / TNS 0.000 / 0 个失败端点 / 11,275 端点、结论行 `All user specified timing constraints are met.`、位流 1,042,208 B @2026-10-12 23:59:27；ILA 版 WNS +0.332 / 0 失败 / 20,165 端点、位流 1,428,064 B @2026-10-13 07:12:11；v11 历史：+0.821 / 11,310 端点、1,086,600 B @21:15:37)**
整体 WHS / THS / 失败端点	**+0.033 ns** / 0.000 / **0**（10582 端点）
WPWS / 失败端点	**9.500 ns** / **0**（3863 端点）
最差路径	`ad_ch2_d[6]` → `u_capture/ch2_meta_reg[6]/D`， `Requirement 20.000`、`Input Delay 24.000`、Data Path **0.907 ns**（logic 0.480 / route 0.427）、`Logic Levels 1 (IBUF)`、DCD 1.595
结论行	**（2026-10-10 v9 历史轮次）`Timing constraints are not met.`**（`impl_timing.rpt:144`）；★ **（2026-10-12 v12）现行结论行已改为 `All user specified timing constraints are met.`**（`sys_clk` WNS **+0.959 ns** / TNS 0.000 / **0 个失败端点** / 11,275 端点， `impl_timing.rpt` @2026-10-12 23:59:16）
DRC	`Checks found: 43`（`DPIP-1` ×11 / `DPOP-1` ×16 / `DPOP-2` ×16，**全部 Warning 且逐条 `Related violations: <none>`**），**0 error / 0 critical**
资源	LUT **6010（28.89%）** / FF **3857（9.27%）** / DSP 16（17.78%）/ Block RAM **0** / Bonded IOB **50**（20.00%）/ BUFGCTRL 1
压缩	`saved 11099520 bits` ⇒ 17,537,696 − 11,099,520 = 6,438,176 bit = **804,772 B**，实测 **804,780 B**（+8 B 头部，与各版口径一致）
位流头	`00 09 0f f0 0f f0 0f f0`（与 08:40 / 12:08 版相同）

⇒ 结论：本版 `sys_clk` 域 **WNS −3.352 ns / TNS −85.116 / 26 个失败端点（全部 ADC 输入端口）/ 10582 端点**， 与 12:08 的 −3.271（§12.3 的"诚实版"）**同源**：根因是当时按 AD9238 的**总输入延迟预算 19.5~24 ns** （**not** 手册 tPD；**手册 `tPD` = 2...6 ns（datasheet 值）**，19.5~24 ns = 3.762 源时钟 + 10.000 半周期 + 3.016 ODDR/OBUF + tPD 2...6 + 1 线缆；且该值是 **v9 的反相相位口径**）写的 `set_input_delay -clock sys_clk -max 24.000 / -min 19.500`（**（2026-10-12 v11）已改为 `14.000/9.500`，并去掉 `create_generated_clock` 的 `-invert`**）， `Requirement 20.000` 减去 24.000 的到达时间，**必然为负** —— 不是逻辑退化，也不是桥引入的问题 （桥只让 LUT +253、FF +329、端点数 9631 → 10582，最差路径仍是 ADC 输入端口）。 ★ **待上板确认的边界项**：`docs/07` §1.8 用示波器量 `tPD_meas`（`J6-26` AD1CLK → `J6-27` CH1_D0）： **≤4 ns 继续用本版**；**≥5 ns 或 `angle_raw` 有毛刺** ⇒ 把 `rtl/ad9238_capture.v:79/:94/:139` 三处 `always @(posedge clk_cap)` 改为 **negedge** 捕获（数据在 19.78–23.78 ns 到达，正好落在下降沿前）， 再重跑生产版并重新签核。 ★ 早期版本（08:40 的 `+0.923 / 0 失败 / 9631 端点 / 962,600 B`、12:08 的 `−3.271 / 1,219,152 B`） **均已被本版取代**；位流大小差异**只看作压缩率差异，不是回归判据。**

临时探针（非交付物，可删）： `_sim_work/` 下的 `probe_*.v`、 `pt_*.v`、 `pz/`、 `pramp/`、 `pin/`、 `pst/`、 `ptrk2/`、 `abzprobe/`、 `loop/`、 `iso/`、 `final/`、 `reg_*`、 `et*`、 `pc*` 等构建目录。

---

13. 工具链踩坑记录（供复现者避坑）

1. **执行策略禁止运行 `.ps1`**： `& .\tools\run_sim.ps1` 报 `无法加载文件...因为在此系统上禁止运行脚本`。绕过：`& powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\tools\run_sim.ps1`。
2. 本机是 **Windows PowerShell 5.1**（`pwsh` 不存在，`$PSHOME` = `C:\Windows\System32\WindowsPowerShell\v1.0`）。 ★ 注意：**harness 的 `pwsh` 工具本身跑在 5.1 上**，所以在它里面调 `pwsh` / `pwsh.exe` 都会 `CommandNotFoundException` —— 必须用 `powershell.exe` 显式拉起子进程。
3. **`.ps1` 必须带 UTF-8 BOM**：5.1 读无 BOM 的 UTF-8 会按 ANSI(GBK) 解码， 中文注释变乱码字节，解析器报**误导性的** `Unexpected token '}'`（行号指向无关的 `}`）。 修法：`[System.IO.File]::WriteAllText($p,$txt,(New-Object System.Text.UTF8Encoding($true)))`。 注意 `write` / `edit` 工具会剥掉 BOM，每次改完 `.ps1` 都要重新加。
4. PowerShell 5.1 的 `Set-Location` **不影响** .NET 的当前目录： `[System.IO.File]::ReadAllText('相对路径')` 仍按进程工作目录解析 —— 用绝对路径。
5. `.NET` 方法调用里避免单引号字符串（会让解析器误判括号配对）。
6. xsim/xvlog 输出按 GBK 解码；日志应以 **UTF-8 带 BOM** 写文件，否则后续读取乱码。
7. `xvlog --sv` 会缓存；输出与源码不符时**换全新目录**重编译。
8. 探针 TB 必须同时打印**激励注入值**与**流水各级**，否则会把"激励是 0"误判成"流水不动"。
9. Python 中文输出在 GBK 控制台乱码，用 `Select-String` 抓 ASCII 关键词即可。